

Criptografia de Chave Pública — do zero, bem básico

Guia de estudo pros vídeos V6 (chave pública), V7 (RSA), V8 (curvas elípticas)

Como ler este guia — Estrutura de Explicabilidade Progressiva (Scaffolding) + Divisão Modular

Cada módulo deste guia é independente e segue **4 camadas** na ordem. Foi exatamente assim que o capítulo de Grupos (Z_7^*) foi construído e validado contigo:

1. **Visão Geral e Contextualização** — o mapa: o que é, por que existe, onde se encaixa.
2. **Analogia Concreta** — tradução da abstração para algo visual/intuitivo (relógio, pista de corrida, ciclo que se repete).
3. **Validação Prática com Números** — pegamos $Z_7^* = \{1, 2, 3, 4, 5, 6\}$ e calculamos passo a passo para o cérebro *ver* a regra funcionar.
4. **Resumo Estruturado (Tabela/Listagem)** — consolidação escaneável para fechar o raciocínio e revisar antes da prova.

Você pode ler um módulo inteiro de cima a baixo, ou pular direto para a Camada 4 para revisar. É o mesmo esqueleto em todos os capítulos densos.

Guia de estudo pros vídeos V6 (chave pública), V7 (RSA), V8 (curvas elípticas). Linguagem simples. Exemplos com números pequenos. Lê de cima pra baixo.

0. A ideia central (leia isso primeiro)

Camada 1 — Visão Geral: Este é o mapa de todo o guia. Se entenderes isto, todo o resto são detalhes.

Cripto simétrica: mesma chave pra fechar e abrir. Problema: como as duas partes combinam a chave sem se encontrar?

Chave pública resolve isso. Cada pessoa tem **duas** chaves:

- **pública** — todo mundo pode ver
- **privada** — só o dono

Regras (decora):

Quero...	Uso
Mandar segredo pra Alice	cifro com a pública da Alice , ela abre com a privada dela
Provar que fui eu que escrevi	assino com a minha privada , todos verificam com a minha pública

Camada 2 — Analogia Concreta: Pensa em **cadeado e chave**. A chave pública é um cadeado aberto que a Alice espalha. Qualquer um tranca uma caixinha com ele. Só a chave privada dela abre. Assinar é o inverso: só você tem o carimbo, todos têm a régua para conferir.

Tudo isso funciona porque existe uma operação **fácil de fazer, difícil de desfazer**:

- multiplicar dois primos é fácil; fatorar o produto é difícil → **RSA**
- calcular $g^x \bmod p$ é fácil; achar x a partir do resultado é difícil → **Diffie-Hellman, ElGamal, DSA**
- somar pontos numa curva elíptica é fácil; achar quantas vezes somou é difícil → **ECC, ECDH, ECDSA**

Isso é “função de mão única” (one-way function). **É a base de tudo.**

Camada 4 — Resumo: Simétrica = 1 chave compartilhada. Assimétrica = par (pública/privada). Cifrar = pública do destinatário. Assinar = privada do autor. Segurança = problema matemático difícil.

1. Fundamentos de Teoria dos Números

Este capítulo é dividido em módulos. O módulo **1.3 (Grupos)** é o exemplar completo do Scaffolding — os demais seguem a mesma lógica de forma mais leve.

1.1 Divisibilidade e primos

Camada 1 — Visão Geral: Vocabulário mínimo para falar de RSA e grupos.

- **$a \mid b$ (“a divide b”)** — existe inteiro k com $b = a \cdot k$. *Exemplo:* $3 \mid 12$ porque $12 = 3 \cdot 4$.
- **Número primo** — inteiro > 1 divisível só por 1 e por ele mesmo. *Exemplos:* 2, 3, 5, 7, 11, 13, ...
- **Teorema fundamental da aritmética** — todo inteiro > 1 se escreve de **um único jeito** como produto de primos. *Exemplo:* $360 = 2^3 \cdot 3^2 \cdot 5$.
- **mdc / gcd** — maior número que divide os dois. *Exemplo:* $\gcd(12, 18) = 6$. Se $\gcd(a, b) = 1 \rightarrow$ são **coprimos**.

Camada 2 — Analogia: Primos são os “átomos” — blocos indivisíveis. O teorema fundamental diz que toda molécula (número) tem uma decomposição atômica única.

Camada 3 — Validação: $\gcd(12, 18)$: divisores de $12 = \{1, 2, 3, 4, 6, 12\}$, de $18 = \{1, 2, 3, 6, 9, 18\} \rightarrow$ maior comum = 6.

Camada 4 — Resumo: $a \mid b \Leftrightarrow b = ak$. Primo = só 1 e ele mesmo. $\gcd=1 \Leftrightarrow$ coprimos (condição para existir inverso).

1.2 Aritmética modular

Camada 1 — Visão Geral: Vamos fazer contas num mundo finito onde tudo dá a volta.

- **$a \bmod n$** — resto da divisão. *Ex:* $17 \bmod 5 = 2$.
- **$a \equiv b \pmod{n}$** — mesmo resto. *Ex:* $17 \equiv 2 \pmod{5}$.

- **O que vale:** soma, subtração, multiplicação funcionam $\rightarrow (a+b) \bmod n$. **Divisão NÃO existe direto** $\rightarrow$ multiplica-se pelo **inverso**.
- **Inverso multiplicativo** — a^{-1} com $a \cdot a^{-1} \equiv 1 \pmod{n}$. *Ex:* $\bmod 7$, $3 \cdot 5 = 15 \equiv 1 \rightarrow$ inverso de 3 é 5. > **Regra de ouro:** existe inverso de $a \bmod n \iff \gcd(a,n)=1$.

Camada 2 — Analogia Concreta (Relógio): $\bmod 12$: 13 horas = 1 hora. O relógio dá a volta, mas a aritmética continua. É por isso que $a+12 \equiv a$.

Camada 3 — Validação com $\mathbb{Z}_7^*$: Vamos testar o inverso na prática: $3 \cdot 5 = 15$. $15 \div 7 = 2$ resto 1 $\rightarrow 15 \equiv 1 \checkmark$. Logo 5 desfaz 3 em $\bmod 7$.

Camada 4 — Resumo:

Conceito	Teste rápido
$a \bmod n$	resto
Inverso existe?	Só se $\gcd(a,n)=1$
Divisão	$a / b = a \cdot b^{-1}$

1.3 Grupo — a estrutura que une tudo [MÓDULO EXEMPLAR COMPLETO]

Este é o módulo onde o Scaffolding foi validado contigo. Use-o como molde para os demais.

Camada 1 — Visão Geral e Contextualização

- **Definição:** um **grupo** é um conjunto G + uma operação $*$ com **4 axiomas**. Pensa como “um conjunto com uma calculadora própria que nunca te joga para fora”.
- **Os 4 axiomas:**
 1. **Fechamento** — $a*b$ continua dentro de G .
 2. **Associatividade** — $(a*b)*c = a*(b*c)$.
 3. **Elemento neutro e** — $a*e = a$ (ex: 0 na soma, 1 na multiplicação).
 4. **Elemento inverso** — todo a tem a^{-1} com $a*a^{-1} = e$.
- **Grupo abeliano:** se ainda vale $a*b = b*a$. Em cripto usamos quase sempre abelianos.
- **Onde se encaixa:**
 - **Álgebra Abstrata** = área geral. **Teoria de Grupos** = sub-tópico. **Aritmética Modular** = grupos com $\bmod n$.
 - *Exemplo da apostila:* $\mathbb{Z}_7^*$ é o **Grupo Multiplicativo de Inteiros Módulo 7 — Grupo Cíclico Finito** (finito = 6 elementos; cíclico = um gerador dá a volta).
 - **Escada: Grupo** (1 operação + 4 regras) $\rightarrow$ **Anel** (2 operações) $\rightarrow$ **Corpo** (anel onde todo $\neq 0$ tem inverso).
- **Por que importa:** DH, ElGamal, ECC são **o mesmo esquema** em grupos diferentes. Entende grupo uma vez, entende todos.

Camada 2 — Analogia Concreta (Pista de Corrida)

- **Ordem do grupo $|G|$:** tamanho da pista — quantas posições existem.
- **Ordem de um elemento a :** número mínimo de passos (a^k) até voltar à largada ($e=1$). É o tamanho do *ciclo* que aquele elemento cria.
- **Em grupo multiplicativo:** a^k . Em grupo aditivo: $k \cdot a = 0$.

Camada 3 — Validação Prática com Números ($\mathbf{Z}_7^* = \{1,2,3,4,5,6\}$, $|\mathbf{G}|=6$)

Ordem do 2 passo a passo: - $2^1=2$, $2^2=4$, $2^3=8 \equiv 1 \pmod{7}$ → chegou no neutro! **ordem=3** → ciclo $2 \rightarrow 4 \rightarrow 1 \rightarrow 2 \dots$

Teorema de Lagrange (desmistificado): ordem de qualquer elemento **divide** $|\mathbf{G}|$. - Teste: $\text{ord}(2)=3$ divide 6? $6 \div 3 = 2$ ✓. $\text{ord}(3)=6$ divide 6? Sim ✓. Nunca existirá ordem 4 em grupo de ordem 6.

Consequência: $a^{|\mathbf{G}|} = e$ sempre. *Teste:* $2^6=(2^3)^2=1^2=1$ ✓

Grupo cíclico e gerador: - Definição: existe g tal que $\mathbf{G} = \{g^1, g^2, \dots, g^{|\mathbf{G}|}=e\}$. - **Prova que 3 é gerador:** $3^1=3$, $3^2=9 \equiv 2$, $3^3=6$, $3^4=18 \equiv 4$, $3^5=12 \equiv 5$, $3^6=15 \equiv 1$ → gerou todos → **Sim, ordem 6**.
- **Contra-exemplo 2:** $2, 4, 1, 2 \dots$ só 3 elementos → **Não é gerador** (ordem $3 \neq 6$).

Camada 4 — Resumo Estruturado (Tabela que você pediu)

Elemento a	Potências até 1	Ordem $\text{ord}(a)$	É gerador?	Comentário
1	1	1	Não	Neutro sempre 1
2	2, 4, 1	3	Não	Ciclo curto
3	3, 2, 6, 4, 5, 1	6	Sim	Gera tudo
4	4, 2, 1	3	Não	Mesmo ciclo do 2
5	5, 4, 6, 2, 3, 1	6	Sim	Outro gerador!
6	6, 1	2	Não	$6 \equiv -1$ sempre ordem 2

Repara: todas as ordens (1,2,3,6) **dividem** 6 — **Lagrange conferido!** Número de geradores = $\varphi(|\mathbf{G}|)$ → $\varphi(6)=2$ → 3 e 5.

1.4 Os dois grupos mod N

Camada 1 — Visão Geral: - $\mathbf{Z}_N$ (aditivo) = $\{0, \dots, N-1\}$ com soma mod N. Sempre grupo, cíclico (gerador 1), tamanho N. - $\mathbf{Z}_N^*$ (multiplicativo) = $\{1, \dots, N-1\}$ coprimos com N com multiplicação mod N. Só coprimos têm inverso.

Camada 2 — Analogia: $\mathbf{Z}_N$ é o relógio com N horas. $\mathbf{Z}_N^*$ é só os ponteiros que conseguem voltar atrás (têm inverso).

Camada 3 — Validação ($\mathbf{Z}_7^*$ vs $\mathbf{Z}_{10}^*$): - $\mathbf{Z}_7^*=\{1,2,3,4,5,6\}$ → 6 elementos (primo, todos entram)
- $\mathbf{Z}_{10}^*=\{1,3,7,9\}$ → 4 elementos (2,4,5,6,8 compartilham fator com 10 e não têm inverso)

Camada 4 — Resumo: $\mathbf{Z}_p^*$ com p primo é **sempre cíclico** → base do Diffie-Hellman.

1.5 Função phi de Euler

Camada 1 — Visão Geral: $\varphi(N)$ = quantos números $1..N$ são coprimos com N = **tamanho de $\mathbf{Z}_N^*$** .

Caso	Fórmula	Exemplo
p primo	$\varphi(p)=p-1$	$\varphi(7)=6$
p, q distintos	$\varphi(pq)=(p-1)(q-1)$	$\varphi(15)=8$

Caso	Fórmula	Exemplo
p^k	$\varphi(p^k) = p^k - p^{k-1}$	$\varphi(8) = 4$

Camada 2 — Analogia: $\varphi(N)$ conta quantos participantes *podem* ter inverso na festa.

Camada 3 — Validação (coração do RSA): Quem sabe p, q calcula $\varphi(N)$ em 1 multiplicação. Quem só tem N precisa fatorar — inviável se N grande.

Camada 4 — Resumo: Segunda linha $(p-1)(q-1)$ é a porta dos fundos do RSA.

1.6 Euler e Fermat

Camada 1 — Visão Geral: Teoremas que permitem “reduzir expoentes”. - **Euler:** se $\gcd(a, N) = 1 \rightarrow a^{\varphi(N)} \equiv 1 \pmod{N}$. - **Fermat (caso primo):** $a^{p-1} \equiv 1 \pmod{p}$.

Camada 2 — Analogia: São atalhos — como saber que dar 100 voltas na pista de 6 posições é igual a dar 4.

Camada 3 — Validação: $3^{100} \bmod 7$: $\varphi(7) = 6$, $100 \bmod 6 = 4 \rightarrow 3^4 = 81 \equiv 4$. Sem teorema faríamos 100 multiplicações.

Camada 4 — Resumo: Permitem RSA desfazer a cifragem: $m^{ed} \equiv m$.

1.7 Teorema Chinês do Resto (CRT) — opcional

Camada 1 — Visão Geral: Se n_1, n_2 coprimos, $x \equiv a_1 \pmod{n_1}$ e $x \equiv a_2 \pmod{n_2}$ tem solução única $\bmod n_1 n_2$.

Camada 2 — Analogia: É como descobrir a hora exata sabendo a hora em dois relógios diferentes.

Camada 3 — Validação (uso RSA): Em vez de calcular $\bmod N = pq$ (2048 bits), calcula $\bmod p$ e $\bmod q$ (1024 bits cada) e recombina $\rightarrow \sim 4\times$ **mais rápido**.

Camada 4 — Resumo: Rápido, mas se não checar o resultado, sofre *fault attack*. Por isso implementações sérias verificam.

2. Teoria dos Números Algorítmica

Scaffolding deste capítulo: Cada algoritmo segue Visão Geral $\rightarrow$ Analogia $\rightarrow$ Conta na mão com números pequenos $\rightarrow$ Tabela de custo.

2.1 Euclides (gcd)

Camada 1 — Visão Geral: $\gcd(a, b) = \gcd(b, a \bmod b)$ até resto 0. $\sim \log(n)$ passos.

Camada 3 — Validação:

```
gcd(252, 198)
252 = 1 · 198 + 54
198 = 3 · 54 + 36
54 = 1 · 36 + 18
36 = 2 · 18 + 0 → gcd = 18
```

Camada 4 — Resumo: Rápido, base de tudo.

2.2 Euclides Estendido $\rightarrow$ inverso

Camada 1 — Visão Geral: Além do gcd, acha x, y com $a \cdot x + b \cdot y = \text{gcd}$. Se $\text{gcd}(a, N) = 1$, então $a \cdot x \equiv 1 \rightarrow x$ é o inverso.

Camada 2 — Analogia: É a “receita” para desfazer a divisão.

Camada 3 — Validação: É assim que RSA calcula $d = e^{-1} \bmod \varphi(N)$. Único jeito prático.

Camada 4 — Resumo: Sem estendido, não há RSA.

2.3 Exponenciação modular (square-and-multiply)

Camada 1 — Visão Geral: Calcular $a^b \bmod n$ sem b multiplicações. Escreve b em binário: quadrado + multiplica se bit=1.

Camada 3 — Validação ($3^{13} \bmod 7$, $13=1101_2$):

bit1 $\rightarrow 1^2 \cdot 3 = 3$

bit1 $\rightarrow 3^2 \cdot 3 = 27 \equiv 6$

bit0 $\rightarrow 6^2 = 36 \equiv 1$

bit1 $\rightarrow 1^2 \cdot 3 = 3 \rightarrow$ resposta 3

Custo: $\sim \log_2(b) \rightarrow 2048$ passos em vez de 2^{2048} . Torna RSA possível.

Camada 2/4 — Analogia e Alerta: Versão ingênua vaza bits por tempo $\rightarrow$ usa Montgomery ladder (tempo constante).

2.4 Teste de primalidade

Camada 1 — Visão Geral: Precisamos gerar primos de 1024+ bits. Testes probabilísticos. - **Fermat:** $a^{n-1} \equiv 1$? Se falhar $\rightarrow$ composto. Se passar $\rightarrow$ provavelmente primo. Fraco (Carmichael 561...). - **Miller–Rabin (usado):** escreve $n-1=2^s \cdot d$. Testa $a^d \equiv 1$ ou $a^{2^r \cdot d} \equiv -1$. Cada rodada erro $\leq 1/4 \rightarrow 40$ rodadas erro $<$ falha de hardware.

Camada 3 — Validação (receita): 1. sorteia ímpar com bit topo 1 $\rightarrow 2$. divide por primos pequenos $\rightarrow 3$. Miller-Rabin k vezes $\rightarrow 4$. falhou volta. Pelo Teorema dos Primos, ~ 1 em 710 tentativas para 1024 bits.

2.5 Ataques — fatoração

Camada 4 — Resumo Estruturado:

Algoritmo	Quando funciona	Custo
Divisão de teste	fatores pequenos	$O(\sqrt{N})$
Pollard $p-1$	$p-1$ só fatores pequenos	rápido
Pollard rho	genérico	$O(N^{1/4})$
Crivo quadrático	N até ~ 100 dígitos	subexp.
NFS	melhor hoje	subexp.

Implicação: escolha p, q grandes, aleatórios, tamanho parecido, $p-1$ não-smooth.

2.6 Ataques — log discreto (DL)

Camada 1 — Visão Geral: Achar x em $g^x=h$.

Algoritmo	Custo	Obs
Força bruta	$O(n)$	
Baby-step giant-step	$O(\sqrt{n})$ tempo e memória	troca
Pollard rho	$O(\sqrt{n})$ tempo, memória ~ 0	usado na prática
Pohlig–Hellman	quebra por fator primo de n	por isso ordem precisa fator primo grande
Index calculus	subexp.	só em $\mathbb{Z}^*_p$, NÃO em curvas

Camada 4 — Resumo: Última linha explica “por que ECC usa chaves menores”.

2.7 Tamanhos de chave recomendados

Camada 4 — Resumo:

Segurança	Simétrico	RSA/DH ($\mathbb{Z}^*_p$)	ECC
112 bits	112	2048	224
128 bits (hoje)	128	3072	256
256 bits	256	15360	512

RSA cresce muito (NFS subexp.), ECC cresce linear ($2\times$ segurança) — só ataque genérico $O(\sqrt{n})$.

3. Hipóteses de Dificuldade (hardness assumptions)

Camada 1 — Visão Geral: Nada é provado seguro. Tudo assume “este problema é difícil”.

- **Fatoração:** dado $N=pq$, achar p, q é inviável.
- **RSA:** dado $N, e, y=x^e$, achar x é inviável (mais forte que fatoração).
- **DL:** dado $h=g^x$, achar x é inviável.
- **CDH:** dados g^a, g^b , calcular $g^{(ab)}$.
- **DDH:** dados g^a, g^b, g^c , decidir se $c=ab$ ou aleatório. **Mais forte que CDH.**

Relação: DDH difícil $\Rightarrow$ CDH difícil $\Rightarrow$ DL difícil.

Camada 2 — Analogia: Quebrar DL quebra tudo; DDH é a hipótese mais exigente (exige indistinguibilidade).

Camada 3 — Validação: Cuidado: DDH é falsa em $\mathbb{Z}^*_p$ inteiro (Legendre vaza 1 bit). Solução: subgrupo de ordem prima q ($p=2q+1$, primo seguro, resíduos quadráticos). Mata Pohlig-Hellman.

Camada 4 — Resumo: Curvas elípticas: mesmo jogo, outro grupo. Sem index calculus $\rightarrow$ chaves $\sim 12\times$ menores.

4. Troca de Chaves

4.1 O problema da distribuição de chaves

Camada 1 — Visão Geral: Se um sistema utiliza exclusivamente criptografia simétrica, cada par de usuários precisa compartilhar uma chave secreta exclusiva. Em uma rede com n participantes, o número total de chaves necessárias cresce quadraticamente:

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

Exemplo: Para $n = 1000$ usuários, são necessárias:

$$\frac{1000 \times 999}{2} = 499.500 \text{ chaves secretas}$$

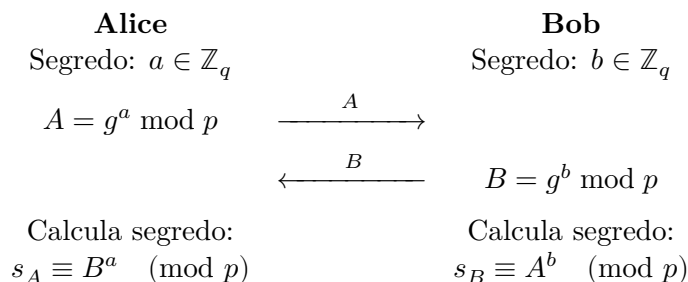
Se cada novo funcionário entrar na empresa, ele precisaria trocar fisicamente uma chave com todos os outros. Isso é logisticamente inviável. Esta foi a motivação histórica que levou Whitfield Diffie, Martin Hellman e Ralph Merkle a proporem o conceito de criptografia de chave pública em 1976.

4.2 Diffie–Hellman Key Exchange (DHKE)

Camada 1 — Visão Geral: O protocolo de Diffie–Hellman permite que duas partes (Alice e Bob), comunicando-se através de um canal totalmente público e monitorado por um espião passivo (Eve), estabeleçam um segredo compartilhado idêntico sem nunca terem se encontrado antes.

Protocolo Formal

1. **Parâmetros Públicos:** Alice e Bob concordam publicamente em um número primo grande p e um gerador g de um subgrupo de $\mathbb{Z}_p^*$ com ordem prima q .
2. **Alice:** escolhe um segredo privado $a \xleftarrow{R} \mathbb{Z}_q$ e calcula sua chave pública $A \equiv g^a \pmod{p}$.
3. **Bob:** escolhe um segredo privado $b \xleftarrow{R} \mathbb{Z}_q$ e calcula sua chave pública $B \equiv g^b \pmod{p}$.



Corretude Matemática: Ambos chegam exatamente ao mesmo valor numérico s , pois as potências comutam:

$$s_A \equiv (g^b \pmod{p})^a \equiv g^{ba} \equiv g^{ab} \equiv (g^a \pmod{p})^b \equiv s_B \pmod{p}$$

Camada 2 — Analogia Concreta (Mistura de Tintas): - Alice e Bob concordam publicamente em uma cor base comum (ex: **amarelo** = g). - Alice escolhe em segredo uma tinta **vermelha** (= a) e a mistura com o amarelo, obtendo **laranja** (= A), que ela envia publicamente a Bob. - Bob escolhe

em segredo uma tinta **azul** ($= b$) e a mistura com o amarelo, obtendo **verde** ($= B$), que ele envia publicamente a Alice. - O espião Eve vê as misturas laranja e verde passando pelo canal, mas é fisicamente inviável separar os pigmentos originais (este é o problema do Logaritmo Discreto). - Alice adiciona seu vermelho secreto à mistura verde de Bob; Bob adiciona seu azul secreto à mistura laranja de Alice. Ambos obtêm exatamente a mesma cor castanha final ($= g^{ab}$).

Camada 3 — Validação Prática com Números Pequenos ($p = 23$, $g = 5$): - Parâmetros públicos: $p = 23$ (primo), $g = 5$. - Alice escolhe segredo privado $a = 6$.

$$A \equiv 5^6 \pmod{23}$$

Como $5^2 = 25 \equiv 2 \pmod{23}$, temos $5^6 = (5^2)^3 \equiv 2^3 = 8 \pmod{23}$. Logo, **Alice envia** $A = 8$. - Bob escolhe segredo privado $b = 15$.

$$B \equiv 5^{15} \pmod{23}$$

Como $5^6 \equiv 8$, temos $5^{12} \equiv 8^2 = 64 \equiv 18 \equiv -5 \pmod{23}$. Então $5^{15} = 5^{12} \times 5^3 \equiv (-5) \times (125 \bmod 23) \equiv (-5) \times 10 = -50 \equiv 19 \pmod{23}$. Logo, **Bob envia** $B = 19$. - **Cálculo do Segredo por Alice:**

$$s_A \equiv B^a \equiv 19^6 \equiv (-4)^6 = 4096 \pmod{23}$$

Dividindo 4096 por 23: $4096 = 23 \times 178 + 2 \implies s_A \equiv 2 \pmod{23}$. - **Cálculo do Segredo por Bob:**

$$s_B \equiv A^b \equiv 8^{15} \pmod{23}$$

Como $8^2 = 64 \equiv 18 \equiv -5 \pmod{23}$, temos $8^4 \equiv (-5)^2 = 25 \equiv 2 \pmod{23}$, $8^8 \equiv 4 \pmod{23}$, $8^{14} \equiv 8^8 \times 8^4 \times 8^2 \equiv 4 \times 2 \times (-5) = -40 \equiv 6 \pmod{23}$. Multiplicando por 8: $s_B \equiv 6 \times 8 = 48 = 2 \times 23 + 2 \equiv 2 \pmod{23}$. - **Conclusão:** Ambos obtiveram $s = 2 \pmod{23}$ com absoluta exatidão!

O papel vital da KDF (Key Derivation Function): O número $s = g^{ab} \bmod p$ **nunca** deve ser usado diretamente como chave simétrica de cifra (ex: AES). Razões: 1. Os elementos de $\mathbb{Z}_p^*$ não possuem distribuição uniforme em strings de bits $\{0, 1\}^{256}$. 2. Se a hipótese DDH vazar resíduos quadráticos, alguns bits de s são previsíveis. **Solução:** Aplica-se uma função de derivação de chave (ex: HKDF com SHA-256):

$$k = \text{HKDF}(s)$$

Camada 4 — Resumo Estruturado:

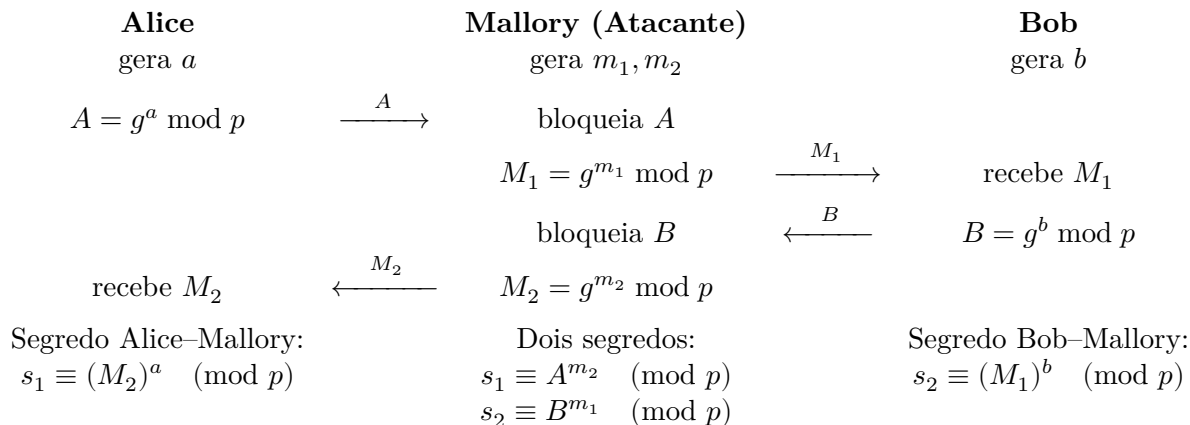
Propriedade	Descrição
O que o atacante passivo (Eve) vê:	$p, g, A = g^a \bmod p, B = g^b \bmod p$
O que Eve quer descobrir:	O segredo $s = g^{ab} \bmod p$
Problema subjacente:	CDH (Computational Diffie-Hellman): calcular g^{ab} a partir de g^a e g^b
Ataque trivial se DL for quebrado:	Achar $a = \log_g A \pmod{p}$, depois calcular $s = B^a \bmod p$
Limitação fundamental:	Não fornece autenticação de origem (vulnerável a MITM)

4.3 Man-in-the-Middle (MITM) — A Fraqueza Mortal do DH Puro

Camada 1 — Visão Geral: O protocolo Diffie–Hellman padrão garante confidencialidade contra espões **passivos** (que apenas escutam o canal), mas é completamente indefeso contra um adversário **ativo** (Mallory), capaz de interceptar, bloquear e alterar mensagens em trânsito.

O Ataque Passo a Passo

Mallory se posiciona entre Alice e Bob e executa **duas trocas DH independentes e simultâneas**: uma fingindo ser Bob para Alice, e outra fingindo ser Alice para Bob.



Resultado do Ataque: 1. Alice cifra mensagens usando a chave simétrica derivada de s_1 . 2. Mallory intercepta o tráfego, decifra com s_1 , lê e altera a mensagem à vontade. 3. Mallory recifra a mensagem adulterada com s_2 e a repassa a Bob. 4. Bob decifra normalmente com s_2 e acredita genuinamente que a mensagem veio de Alice.

Camada 2 — Analogia: É como um tradutor falso sentado entre dois diplomatas que falam línguas diferentes: o tradutor escuta Alice, anota os segredos, inventa o que quiser e repassa para Bob com voz convincente.

Camada 3 — Validação: O ataque funciona porque Alice recebe M_2 e não tem **nenhuma forma de verificar** se aquele valor g^{m_2} foi de fato gerado por Bob ou por um terceiro. Números avulsos não carregam identidade.

Camada 4 — Resumo e Conserto Definitivo: - **Regra de ouro:** Diffie–Hellman puro estabelece canal cifrado, mas **não autentica as pontas**. - **Solução no mundo real:** Chaves públicas efêmeras de DH precisam ser **assinadas digitalmente** por uma autoridade confiável através de certificados digitais (X.509) — esta é a fundação do protocolo **TLS 1.3** utilizado no HTTPS.

5. Cifragem de Chave Pública

5.1 Sintaxe e Noções de Segurança Formal

Camada 1 — Visão Geral: Um esquema de cifragem de chave pública (ou assimétrica) é uma tupla de três algoritmos em tempo polinomial probabilístico (Gen, Enc, Dec):

1. $\text{Gen}(1^\lambda) \rightarrow (\text{pk}, \text{sk})$: Algoritmo probabilístico que recebe o parâmetro de segurança λ e gera um par de chaves: a chave pública pk (distribuída livremente) e a chave privada sk (mantida em segredo absoluto).
2. $\text{Enc}(\text{pk}, m) \rightarrow c$: Algoritmo (geralmente probabilístico) que usa a chave pública pk para transformar a mensagem clara $m \in \mathcal{M}$ em um texto cifrado $c \in \mathcal{C}$.
3. $\text{Dec}(\text{sk}, c) \rightarrow m$: Algoritmo determinístico que usa a chave privada sk para recuperar a mensagem original m . Se c for inválido, retorna um símbolo de erro $\perp$.

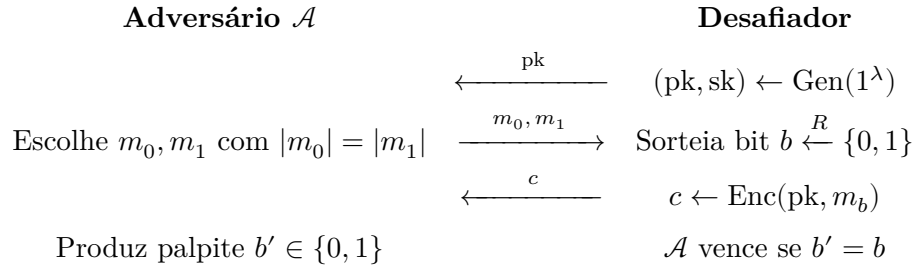
Corretude: Para todo par gerado $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$ e toda mensagem $m \in \mathcal{M}$:

$$\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, m)) = m$$

O Experimento CPA (Chosen-Plaintext Attack)

Na criptografia assimétrica, **todo adversário tem capacidade de realizar ataques de texto claro escolhido (CPA)** por definição, pois ele possui a chave pública pk e pode cifrar quantas mensagens desejar por conta própria.

A segurança semântica contra CPA é modelada pelo seguinte jogo interativo entre o Adversário $\mathcal{A}$ e um Desafiador (Challenger):



Definição: Um esquema é **CPA-seguro** (indistinguível sob ataque por texto claro escolhido) se, para todo adversário PPT $\mathcal{A}$, a probabilidade de vitória satisfaz:

$$\Pr[\mathcal{A} \text{ vence}] \leq \frac{1}{2} + \text{negl}(\lambda)$$

Teorema Fundamental: Nenhuma cifra determinística de chave pública pode ser CPA-segura.

Demonstração: Se Enc for determinística, o adversário $\mathcal{A}$, ao receber o cifrado desafio c , simplesmente calcula por conta própria:

$$c_0 = \text{Enc}(\text{pk}, m_0)$$

Se $c = c_0$, ele responde com certeza $b' = 0$; caso contrário, responde $b' = 1$. O adversário vence com probabilidade 1 (vantagem = 1/2). **Conclusão:** Todo esquema de chave pública precisa ser **probabilístico (randomizado)**.

O Experimento CCA (Chosen-Ciphertext Attack / IND-CCA2)

Em cenários reais, o adversário frequentemente consegue induzir o sistema a decifrar mensagens modificadas (por exemplo, analisando mensagens de erro de rede, códigos de status ou respostas de servidores). Isso é capturado pelo modelo **CCA**:

1. O adversário recebe pk e ganha acesso a um **Oráculo de Decifragem** $\mathcal{O}_{\text{Dec}}(\cdot)$, podendo solicitar a decifragem de quaisquer textos cifrados c_i que desejar.
2. O adversário escolhe m_0, m_1 e recebe o desafio $c^* = \text{Enc}(pk, m_b)$.
3. O adversário continua com acesso ao oráculo $\mathcal{O}_{\text{Dec}}(\cdot)$ para qualquer cifrado c' , **com a única restrição de que $c' \neq c^*$** .
4. $\mathcal{A}$ vence se adivinhar o bit b .

O perigo da Maleabilidade: Se um esquema permitir que o adversário transforme $c = \text{Enc}(pk, m)$ em um novo cifrado válido $c' = \text{Enc}(pk, 2m)$ sem conhecer m , o esquema é **maleável**. O adversário simplesmente envia c' ao oráculo de decifragem, recebe $2m$, divide por 2 e descobre m com sucesso trivial. **Exigência prática:** Todo sistema de produção exige segurança **IND-CCA2 (Não-Maleável)**.

5.2 Cifragem Híbrida (KEM-DEM)

Camada 1 — Visão Geral: Criptografia de chave pública possui duas limitações severas na prática:

1. **Desempenho:** É entre 100 e 1000 vezes mais lenta que a criptografia simétrica.
2. **Capacidade:** O tamanho da mensagem m é estritamente limitado pelo tamanho do módulo (ex: no RSA-2048, não é possível cifrar mais do que 245 bytes diretamente).

Para resolver isso, toda a criptografia moderna de chave pública (TLS, Signal, PGP, SSH) adota a **Cifragem Híbrida**, decomposta em dois módulos independentes:

```
+-----+
| Cifragem HÍBRIDA                                     |
| |                                                     |
| 1. KEM (Key Encapsulation Mechanism)                 |
|   Gera chave simétrica aleatória  $k \leftarrow \{0,1\}^{256}$  |
|   Cifra  $k$  com chave pública do destinatário:  $c_{\text{kem}} = \text{Enc}_{\text{pk}}(k)$  |
| |                                                     |
| 2. DEM (Data Encapsulation Mechanism)                 |
|   Cifra os dados longos  $M$  usando  $k$  com cifra simétrica rápida: |
|    $c_{\text{dem}} = \text{AES-256-GCM}_k(M)$  |
| |                                                     |
| Pacote final transmitido:  $C = (c_{\text{kem}}, c_{\text{dem}})$  |
+-----+
```

Decifragem: 1. O destinatário usa sua chave privada sk no KEM para decifrar c_{kem} e recuperar a chave simétrica k . 2. Usa k no DEM para decifrar e validar a integridade de c_{dem} via AES-GCM.

Camada 4 — Resumo: Se o KEM for IND-CCA2 e o DEM for AEAD (cifra autenticada), o sistema híbrido resultante é provadamente IND-CCA2. O padrão moderno que formaliza essa arquitetura é o **HPKE (Hybrid Public Key Encryption — RFC 9180)**.

5.3 Criptossistema ElGamal

Camada 1 — Visão Geral: Proposto por Taher Elgamal em 1985, adapta o princípio de troca de chaves Diffie–Hellman diretamente para a cifragem de mensagens.

Algoritmo Formal

1. **Geração de Chaves (Gen):**

- Escolhe-se um grupo cíclico G de ordem prima q com gerador g , no qual o problema DDH seja difícil (por exemplo, um subgrupo de $\mathbb{Z}_p^*$).
- Escolhe-se o segredo privado $x \xleftarrow{R} \mathbb{Z}_q$.
- Calcula-se a chave pública $h \equiv g^x \pmod{p}$.
- **Chave pública:** (p, q, g, h)
- **Chave privada:** x

2. **Cifragem (Enc):**

- Para cifrar uma mensagem $m \in G$ usando a chave pública h :
- Sorteia-se um **nonce efêmero** aleatório $r \xleftarrow{R} \mathbb{Z}_q$.
- O texto cifrado é o par (c_1, c_2) dado por:

$$c_1 \equiv g^r \pmod{p}, \quad c_2 \equiv m \cdot h^r \pmod{p}$$

3. **Decifragem (Dec):**

- O receptor usa sua chave privada x para remover a máscara:

$$s \equiv c_1^x \pmod{p}$$

- Multiplica c_2 pelo inverso multiplicativo modular de s :

$$m \equiv c_2 \cdot (c_1^x)^{-1} \pmod{p}$$

Demonstração Algébrica da Corretude

Por que a decifragem recupera exatamente m ? Substituindo as definições de c_1 e c_2 :

$$\frac{c_2}{c_1^x} \equiv \frac{m \cdot h^r}{(g^r)^x} \equiv \frac{m \cdot (g^x)^r}{g^{rx}} \equiv \frac{m \cdot g^{xr}}{g^{rx}} \equiv m \cdot 1 \equiv m \pmod{p}$$

Como a operação ocorre em um corpo finito $\mathbb{F}_p$, a “divisão” é rigorosamente executada multiplicando pelo inverso modular $(c_1^x)^{-1} \pmod{p}$.

Validação com Números Pequenos Passo a Passo

Vamos validar com $p = 23$, $g = 5$, cuja ordem é $q = 22$.

• **Setup de Chaves de Bob:**

- Bob escolhe chave privada $x = 6$.
- Calcula chave pública: $h \equiv 5^6 \pmod{23}$.

$$5^2 = 25 \equiv 2 \pmod{23} \implies 5^6 = (5^2)^3 \equiv 2^3 = 8 \pmod{23}$$

Logo, **chave pública** $h = 8$.

- **Alice Cifra a Mensagem $m = 10$:**

- Alice escolhe nonce efêmero aleatório $r = 3$.

- Calcula c_1 :

$$c_1 \equiv g^r \equiv 5^3 = 125 = 5 \times 23 + 10 \equiv 10 \pmod{23}$$

- Calcula a máscara compartilhada:

$$h^r \equiv 8^3 = 512 = 22 \times 23 + 6 \equiv 6 \pmod{23}$$

- Calcula c_2 :

$$c_2 \equiv m \cdot h^r \equiv 10 \times 6 = 60 = 2 \times 23 + 14 \equiv 14 \pmod{23}$$

- Alice envia a Bob o par cifrado: $(c_1, c_2) = (10, 14)$.

- **Bob Decifra com sua Chave Privada $x = 6$:**

1. Bob calcula a máscara a partir de c_1 :

$$s \equiv c_1^x \equiv 10^6 \pmod{23}$$

Calculando potências de 10 em mod23:

$$10^2 = 100 = 4 \times 23 + 8 \equiv 8 \pmod{23}$$

$$10^6 = (10^2)^3 \equiv 8^3 = 512 \equiv 6 \pmod{23}$$

Bob descobre que a máscara compartilhada vale $s = 6$.

2. Bob encontra o inverso modular $6^{-1} \pmod{23}$: Procuramos y tal que $6y \equiv 1 \pmod{23}$.

$$6 \times 4 = 24 = 23 + 1 \equiv 1 \pmod{23} \implies 6^{-1} \equiv 4 \pmod{23}$$

3. Bob recupera a mensagem original:

$$m \equiv c_2 \cdot 6^{-1} \equiv 14 \times 4 = 56 = 2 \times 23 + 10 \equiv 10 \pmod{23}$$

A mensagem decifrada é exatamente $m = 10$!

Análise de Segurança e Fraquezas de ElGamal

1. **Segurança CPA:** ElGamal é semanticamente seguro (IND-CPA) se e somente se a hipótese **DDH (Decisional Diffie-Hellman)** for verdadeira no grupo G . Como a cada cifragem sorteia-se um novo r , a mesma mensagem cifrada duas vezes produz textos cifrados completamente diferentes.
2. **Expansão de Texto:** O texto cifrado consiste em dois elementos de grupo (c_1, c_2) , tendo o dobro do tamanho da mensagem clara ($2\times$).
3. **Maleabilidade (Inseguro contra CCA):** Dado um cifrado válido $(c_1, c_2) = (g^r, m \cdot h^r)$, qualquer pessoa pode multiplicar a segunda coordenada por uma constante $\alpha \in G$:

$$(c_1, c'_2) = (c_1, \alpha \cdot c_2) = (g^r, (\alpha \cdot m) \cdot h^r)$$

Este novo par é um texto cifrado perfeitamente válido para a mensagem $\alpha \cdot m \pmod{p}$. Logo, **ElGamal é maleável e NÃO é IND-CCA2**.

4. **O Desastre da Reutilização do Nonce r :** Se o remetente cifrar duas mensagens m_1 e m_2 usando o mesmo nonce r :

$$c_2^{(1)} \equiv m_1 \cdot h^r \pmod{p}, \quad c_2^{(2)} \equiv m_2 \cdot h^r \pmod{p}$$

O atacante divide as duas cifras públicas:

$$\frac{c_2^{(1)}}{c_2^{(2)}} \equiv \frac{m_1 \cdot h^r}{m_2 \cdot h^r} \equiv \frac{m_1}{m_2} \pmod{p}$$

Se o atacante conhecer ou adivinhar m_1 , ele calcula m_2 imediatamente sem chave privada!

5.4 RSA de Livro-Texto (Textbook RSA)

Camada 1 — Visão Geral: Criado por Ron Rivest, Adi Shamir e Leonard Adleman em 1977, baseia-se na assimetria computacional entre multiplicar primos e fatorar o produto resultante.

Geração de Chaves Passo a Passo

1. Escolhem-se aleatoriamente dois números primos grandes e distintos p e q de mesmo tamanho de bits (ex: 1024 bits cada).
2. Calcula-se o módulo composto:

$$N = p \cdot q$$

3. Calcula-se a função totiente de Euler de N :

$$\varphi(N) = (p-1)(q-1)$$

4. Escolhe-se um expoente público e tal que:

$$\gcd(e, \varphi(N)) = 1 \quad \text{e} \quad 1 < e < \varphi(N)$$

Padrão universal da indústria: $e = 65537 = 2^{16} + 1$ (possui apenas dois bits '1' na representação binária, permitindo square-and-multiply ultrarrápido).

5. Calcula-se o expoente privado d como o inverso multiplicativo de e módulo $\varphi(N)$:

$$e \cdot d \equiv 1 \pmod{\varphi(N)}$$

(Calculado eficientemente através do Algoritmo Estendido de Euclides).

- **Chave pública:** $(pk) = (N, e)$
 - **Chave privada:** $(sk) = (N, d)$ ou diretamente os fatores (p, q)
-

Operações

- **Cifragem:** Para $m \in \{0, \dots, N-1\}$:

$$c \equiv m^e \pmod{N}$$

- **Decifragem:** Para recuperar m :

$$m \equiv c^d \pmod{N}$$

Demonstração da Corretude com o Teorema de Euler

Como $e \cdot d \equiv 1 \pmod{\varphi(N)}$, existe um número inteiro k tal que:

$$e \cdot d = 1 + k \cdot \varphi(N)$$

Ao decifrar o texto cifrado:

$$c^d \equiv (m^e)^d \equiv m^{ed} \equiv m^{1+k\varphi(N)} \equiv m \cdot (m^{\varphi(N)})^k \pmod{N}$$

Pelo **Teorema de Euler**, para qualquer m coprimo com N ($\gcd(m, N) = 1$):

$$m^{\varphi(N)} \equiv 1 \pmod{N}$$

Substituindo na equação:

$$c^d \equiv m \cdot (1)^k \equiv m \pmod{N}$$

(Nota: mesmo se $\gcd(m, N) \neq 1$, a igualdade ainda se sustenta identicamente usando o Teorema Chinês do Resto módulo p e módulo q).

Validação com Números Pequenos Passo a Passo

- **Setup de Parâmetros:**

- Primos: $p = 3, q = 11$.
- Módulo: $N = 3 \times 11 = 33$.
- Totiente: $\varphi(N) = (3 - 1)(11 - 1) = 2 \times 10 = 20$.
- Expoente público: $e = 3$. Verificação: $\gcd(3, 20) = 1 \checkmark$.
- Expoente privado d : procuramos d tal que $3d \equiv 1 \pmod{20}$.

$$3 \times 7 = 21 = 20 + 1 \equiv 1 \pmod{20} \implies d = 7$$

- Par público: $(N = 33, e = 3)$. Segredo privado: $d = 7$.

- **Cifragem de $m = 4$:**

$$c \equiv m^e \equiv 4^3 = 64 \pmod{33}$$

Dividindo 64 por 33: $64 = 1 \times 33 + 31 \implies c = 31$.

- **Decifragem de $c = 31$:**

$$m \equiv c^d \equiv 31^7 \pmod{33}$$

Simplificando a base antes de exponenciar:

$$31 \equiv -2 \pmod{33}$$

$$31^7 \equiv (-2)^7 = -128 \pmod{33}$$

Dividindo -128 por 33:

$$-128 = (-4) \times 33 + 4 \implies m \equiv 4 \pmod{33}$$

A mensagem recuperada foi exatamente $m = 4$!

Por que o RSA de Livro-Texto é Frequentemente Fatal em Sistemas Reais?

O RSA cru (“textbook”) **nunca** deve ser usado em nenhum sistema de produção devido a 4 vulnerabilidades devastadoras:

1. **É Totalmente Determinístico:** A mesma mensagem m sempre gera o mesmo cifrado c . Isso destrói a segurança semântica: o atacante pode testar palpites triviais (ex: cifrar “SIM” e “NÃO” e comparar com o tráfego interceptado).
2. **Homomorfismo Multiplicativo e Maleabilidade:** Sejam $c_1 \equiv m_1^e \pmod{N}$ e $c_2 \equiv m_2^e \pmod{N}$. Então:

$$c_1 \cdot c_2 \equiv (m_1 \cdot m_2)^e \pmod{N}$$

Ataque prático de manipulação: Um atacante intercepta $c \equiv m^e \pmod{N}$. Ele escolhe um fator arbitrário s e calcula:

$$c' \equiv c \cdot s^e \equiv (m \cdot s)^e \pmod{N}$$

Ao enviar c' para a vítima decifrar, o resultado é $m \cdot s \pmod{N}$. Se o atacante souber que c era uma proposta monetária em um contrato, ele pode dobrar o valor ($s = 2$) sem precisar conhecer a chave privada nem saber quanto valia a proposta original!

3. **Ataque da Raiz Direta ($m^e < N$):** Se o expoente público for $e = 3$ e a mensagem m for curta (ex: $m = 100$), temos $m^3 = 1.000.000$. Se N tiver 2048 bits ($N \approx 10^{616}$), a operação $m^3 \pmod{N}$ **não sofre redução modular** ($m^e < N$). O atacante calcula a raiz cúbica ordinária nos números reais:

$$m = \sqrt[3]{c}$$

e quebra o segredo instantaneamente!

4. **Ataque de Håstad (Broadcast com $e = 3$):** Se a mesma mensagem m for enviada para 3 pessoas diferentes que usam $e = 3$ com módulos N_1, N_2, N_3 coprimos:

$$c_1 \equiv m^3 \pmod{N_1}, \quad c_2 \equiv m^3 \pmod{N_2}, \quad c_3 \equiv m^3 \pmod{N_3}$$

Usando o Teorema Chinês do Resto, o atacante encontra $C \equiv m^3 \pmod{N_1 N_2 N_3}$. Como $m < N_i$, temos $m^3 < N_1 N_2 N_3$, logo nenhuma redução modular ocorreu no módulo combinado. Basta extrair a raiz cúbica inteira $\sqrt[3]{C}$ para achar m .

5.5 O Padding PKCS #1 v1.5 e o Ataque de Bleichenbacher

Para impedir os ataques ao RSA cru, desenvolveu-se o padrão **PKCS #1 v1.5**, que insere bytes de preenchimento (padding) aleatório na mensagem antes de elevar a e .

Estrutura do Bloco Formatado (para módulo de k bytes)

0x00 (1 byte)	0x02 (1 byte)	Bytes Aleatórios $PS \neq 0x00$ (pelo menos 8 bytes)	0x00 (1 byte)	Dados da Mensagem M (tamanho restante)
------------------	------------------	---	------------------	---

- O primeiro byte 0x00 garante que o número resultante como inteiro seja menor que o módulo N .
- O byte 0x02 indica que o bloco é formatado para cifragem de chave pública.
- A cadeia de preenchimento PS adiciona aleatoriedade indispensável para garantir que o esquema seja probabilístico.
- Um bloco cuja decifragem produza exatamente esse formato é chamado de **PKCS-conforme**.

Em um módulo de k bytes ($N \approx 2^{8k}$), a exigência de que o bloco comece com `0x00 0x02` restringe o valor inteiro de m ao intervalo:

$$2B \leq m < 3B, \quad \text{onde } B = 2^{8(k-2)}$$

O Ataque do Milhão de Mensagens (Bleichenbacher 1998)

Daniel Bleichenbacher descobriu que se um servidor de aplicação decifrar um texto cifrado c e retornar códigos de erro distintos dependendo de o padding ser válido ou não (ex: erro "Bad Padding" vs erro de processamento interno), o servidor atua como um **Oráculo de Padding**:

$$\mathcal{O}(c) = \begin{cases} 1 & \text{se } c^d \bmod N \in [2B, 3B - 1] \text{ (padding válido)} \\ 0 & \text{caso contrário (padding corrompido)} \end{cases}$$

Como o atacante explora esse oráculo: 1. O atacante intercepta o cifrado alvo $c = m^e \bmod N$. 2. Ele escolhe inteiros s_i e calcula cifrados manipulados:

$$c_i \equiv c \cdot (s_i)^e \equiv (m \cdot s_i)^e \pmod{N}$$

3. Ele envia c_i ao servidor. Se o servidor responder que o padding foi aceito ($\mathcal{O}(c_i) = 1$), o atacante sabe com certeza matemática que:

$$2B \leq (m \cdot s_i \bmod N) < 3B$$

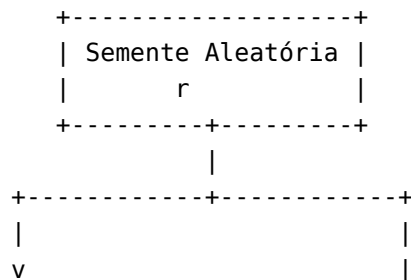
4. Cada resposta afirmativa do servidor elimina vastas faixas numéricas de onde m pode estar localizado. Com um algoritmo inteligente de busca e refinamento de intervalos, o atacante descobre a mensagem inteira m com cerca de algumas centenas de milhares a um milhão de consultas. 5. Em 2017, este ataque foi redescoberto afetando dezenas de grandes fabricantes de servidores sob o nome de **ROBOT (Return Of Bleichenbacher's Oracle Threat)**.

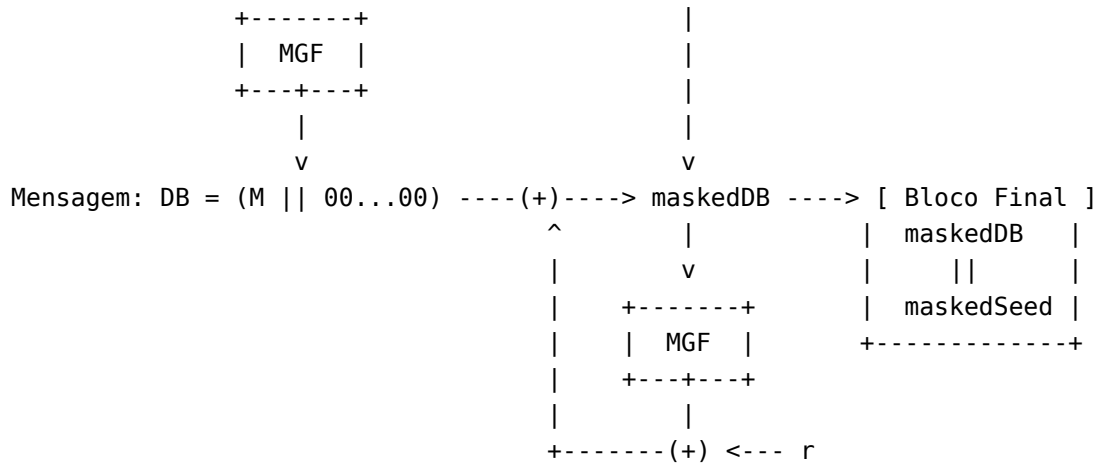
5.6 RSA-OAEP (Optimal Asymmetric Encryption Padding)

Para sanar de forma definitiva as falhas do PKCS #1 v1.5, Mihir Bellare e Phillip Rogaway propuseram o **OAEP**, padronizado no PKCS #1 v2.0 e utilizado obrigatoriamente em todos os padrões modernos.

A Estrutura de Feistel de 2 Rodadas

O OAEP combina a mensagem M com uma semente verdadeiramente aleatória r através de duas funções de geração de máscara (MGF_1) baseadas em funções de hash criptográficas (como SHA-256):





1. **Criação do Bloco de Dados:** Concatena-se a mensagem com bytes fixos de verificação: $DB = M \parallel 00 \dots 00$.
2. **Primeira Máscara:** Gera-se uma semente criptográfica aleatória r . Aplica-se a função de máscara $MGF(r)$ e faz-se o XOR com o bloco de dados:

$$\text{maskedDB} = DB \oplus MGF(r)$$

3. **Segunda Máscara:** Aplica-se a função de máscara sobre maskedDB e mascara-se a semente:

$$\text{maskedSeed} = r \oplus MGF(\text{maskedDB})$$

4. **Bloco Cifrado com RSA:** O texto claro formatado é a concatenação $EM = \text{maskedDB} \parallel \text{maskedSeed}$, que é então elevado a $e \pmod{N}$.

Por que o OAEP é Inviolável contra CCA?

- **Efeito Avalanche Total:** Qualquer modificação de um único bit no texto cifrado se propaga caoticamente por todo o bloco durante as etapas de desmascaramento.
- **Falha com Probabilidade Praticamente 1:** Se o bloco for adulterado, os bytes de verificação 00...00 finais **não** serão recuperados, e a decifragem rejeita o pacote instantaneamente.
- **Segurança Provada:** Bellare e Rogaway provaram que o RSA-OAEP é **IND-CCA2 seguro** no Modelo de Oráculo Aleatório (ROM).
- **Implementação em Tempo Constante:** A rejeição de blocos inválidos é calculada sem desvios de código ou mensagens de erro reveladoras, erradicando canais laterais (timing attacks).

6. Assinaturas Digitais

6.1 Sintaxe e Segurança Formal

Camada 1 — Visão Geral: Uma assinatura digital é o equivalente criptográfico da assinatura manuscrita em papel, porém matematicamente infalsificável. É definida por uma tupla de três algoritmos (Gen, Sign, Verify):

1. $\text{Gen}(1^\lambda) \rightarrow (\text{pk}, \text{sk})$: Gera o par de chaves. A chave privada sk é usada exclusivamente pelo autor para assinar; a chave pública pk é disponibilizada a todos para verificar.

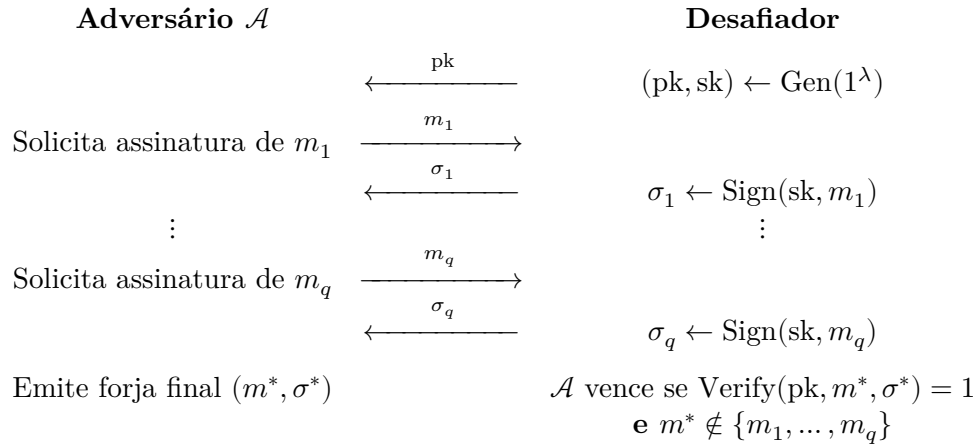
2. $\text{Sign}(\text{sk}, m) \rightarrow \sigma$: Recebe a chave privada sk e a mensagem m , gerando a assinatura digital σ .
3. $\text{Verify}(\text{pk}, m, \sigma) \rightarrow \{0, 1\}$: Algoritmo determinístico que recebe a chave pública pk , a mensagem m e a assinatura σ . Retorna 1 se a assinatura for autêntica e válida, ou 0 caso contrário.

Corretude: Para todo $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$ e toda mensagem m :

$$\text{Verify}(\text{pk}, m, \text{Sign}(\text{sk}, m)) = 1$$

O Experimento EUF-CMA (Existential Unforgeability under Chosen-Message Attack)

O padrão universal de segurança para assinaturas digitais exige que nenhum adversário seja capaz de forjar uma assinatura válida para **nenhuma mensagem nova**, mesmo tendo a capacidade de solicitar assinaturas para mensagens de sua própria escolha:



Definição: Um esquema é **EUF-CMA seguro** se a probabilidade de vitória de qualquer adversário PPT for desprezível ($\Pr[\mathcal{A} \text{ vence}] \leq \text{negl}(\lambda)$).

A Diferença Crucial: MAC (Simétrico) vs. Assinatura Digital (Assimétrica)

Propriedade	MAC (ex: HMAC-SHA256)	Assinatura Digital (ex: ECDSA, Ed25519)
Chaves:	Simétrica: 1 única chave secreta k compartilhada	Assimétrica: par (pk, sk)
Quem cria a tag/assinatura:	Qualquer um que saiba k (Alice ou Bob)	Exclusivamente o dono da chave privada sk
Quem verifica:	Apenas quem sabe k	Qualquer pessoa no mundo que tenha a chave pública pk
Autenticidade e Integridade:	Sim	Sim
Não-Repúdio (Procuração Legal / Juiz):	NÃO. Como Bob conhece k , ele poderia ter forjado o MAC e acusado Alice.	SIM. Apenas Alice possui sk ; Bob pode apresentar a mensagem assinada a um juiz imparcial.

6.2 O Paradigma Hash-and-Sign

Na prática, nunca se assina a mensagem m diretamente, pois operações de chave pública são computacionalmente custosas para dados grandes e introduzem vulnerabilidades estruturais. O padrão universal é **Hash-and-Sign**:

$$\sigma = \text{Sign}(\text{sk}, H(m))$$

onde $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ é uma função de hash criptográfica segura (como SHA-256 ou SHA-3).

Por que a Resistência à Colisão é Estritamente Obrigatória?

Para funções de hash, existem três propriedades de segurança: 1. **Resistência à Pré-imagem**: Dado y , é difícil achar m tal que $H(m) = y$. 2. **Resistência à Segunda Pré-imagem**: Dado m_1 , é difícil achar $m_2 \neq m_1$ com $H(m_1) = H(m_2)$. 3. **Resistência à Colisão**: É computacionalmente inviável encontrar **qualquer** par arbitrário $m_1 \neq m_2$ tal que $H(m_1) = H(m_2)$.

Ataque do Aniversário contra Hash-and-Sign: Se uma função hash não for resistente à colisão, um adversário (Eve) pode quebrar o sistema facilmente: 1. Eve encontra computacionalmente dois documentos diferentes m_1 e m_2 com o mesmo hash:

$$H(m_1) = H(m_2)$$

- m_1 é um contrato legítimo inofensivo: “*Declaro que recebi R\$ 10,00 de Eve*”. - m_2 é um contrato fraudulento devastador: “*Transfiro todos os meus bens para Eve*”. 2. Eve apresenta m_1 a Alice e solicita que ela assine digitalmente. 3. Alice assina o hash de m_1 :

$$\sigma = \text{Sign}(\text{sk}_A, H(m_1))$$

4. Como $H(m_1) = H(m_2)$, a assinatura σ produzida por Alice é **identicamente válida para o contrato m_2** :

$$\text{Verify}(\text{pk}_A, m_2, \sigma) = 1$$

5. Eve anexa σ ao contrato fraudulento m_2 e executa a cobrança legal!

Casos Reais Históricos: - **MD5**: Em 2008, pesquisadores criaram uma Autoridade Certificadora falsa devido a colisões no MD5. Em 2012, o malware de ciberguerra **Flame** usou colisões MD5 para forjar assinaturas da Microsoft no Windows Update. - **SHA-1**: Quebrado pelo ataque SHAttered em 2017 pelo CWI e Google. **Regra de Produção**: Utilize exclusivamente **SHA-256, SHA-384, SHA-512 ou SHA-3**.

6.3 Assinaturas RSA

O Esquema RSA Cru (“Textbook”) e suas Falhas Fatais

No RSA cru, assinar é exponenciar com a chave privada d , e verificar é exponenciar com a pública e :

$$\sigma \equiv m^d \pmod{N} \quad \implies \quad \text{Verificação: } \sigma^e \equiv (m^d)^e \equiv m \pmod{N}$$

Vulnerabilidade 1 — Forja Existencial Trivial (sem conhecer d): O atacante escolhe qualquer valor numérico arbitrário $\sigma \in \mathbb{Z}_N^*$. Ele calcula a “mensagem” correspondente usando a chave pública:

$$m \equiv \sigma^e \pmod{N}$$

O par (m, σ) é imediatamente aceito por qualquer verificador como uma assinatura autêntica, sem que o atacante saiba a chave privada!

Vulnerabilidade 2 — Homomorfismo Multiplicativo: Se o atacante obtiver assinaturas legítimas de Alice para duas mensagens distintas m_1 e m_2 :

$$\sigma_1 \equiv m_1^d \pmod{N}, \quad \sigma_2 \equiv m_2^d \pmod{N}$$

Ele pode forjar a assinatura para a mensagem composta $m^* = m_1 \cdot m_2 \pmod{N}$ multiplicando as duas assinaturas:

$$\sigma^* \equiv \sigma_1 \cdot \sigma_2 \equiv m_1^d \cdot m_2^d \equiv (m_1 \cdot m_2)^d \pmod{N}$$

A Solução Moderna: RSA-PSS (Probabilistic Signature Scheme)

Padronizado no PKCS #1 v2.1 (RFC 8017), o **RSA-PSS** elimina completamente a maleabilidade:

1. Adiciona um valor aleatório (*salt*) à mensagem antes do hash.
2. Aplica funções de geração de máscara baseadas em Feistel (*MGF*).
3. É **provadamente EUF-CMA seguro** no Modelo de Oráculo Aleatório (ROM).

6.4 DSA e ECDSA (Do Zero, Passo a Passo)

O algoritmo **DSA (Digital Signature Algorithm)** e sua versão moderna em curvas elípticas, **ECDSA (Elliptic Curve DSA)**, baseiam-se na dificuldade do logaritmo discreto.

Parâmetros Globais do ECDSA

- Uma curva elíptica $E(\mathbb{F}_p)$ definida sobre um corpo finito $\mathbb{F}_p$.
- Um ponto base gerador $G \in E(\mathbb{F}_p)$ com ordem prima n grande (isto é, $n \cdot G = \mathcal{O}$).
- **Chave Privada:** Um número inteiro secreto $d \xleftarrow{R} \{1, \dots, n-1\}$.
- **Chave Pública:** O ponto correspondente na curva:

$$Q = d \cdot G$$

Algoritmo de Assinatura ECDSA

Para assinar uma mensagem m com a chave privada d :

1. Calcula-se o hash da mensagem: $e = H(m)$ (trunca-se e para o tamanho em bits de n).
2. Sorteia-se um **nonce efêmero estritamente secreto e uniforme**:

$$k \xleftarrow{R} \{1, \dots, n-1\}$$

3. Calcula-se o ponto resultante da multiplicação escalar na curva:

$$R = k \cdot G = (x_R, y_R)$$

4. A primeira coordenada da assinatura é a abscissa do ponto reduzida no módulo da ordem do grupo:

$$r \equiv x_R \pmod{n}$$

(Se $r = 0$, volta-se ao passo 2 e sorteia-se um novo k).

5. A segunda coordenada da assinatura é calculada como:

$$s \equiv k^{-1}(e + r \cdot d) \pmod{n}$$

(Se $s = 0$, volta-se ao passo 2 e sorteia-se um novo k).

6. A assinatura digital final é o par de números inteiros:

$$\sigma = (r, s)$$

Algoritmo de Verificação ECDSA

Para verificar a assinatura (r, s) da mensagem m usando a chave pública Q :

1. Valida-se se r e s são inteiros no intervalo válido: $1 \leq r, s \leq n - 1$.
2. Calcula-se o hash $e = H(m)$.
3. Calcula-se o inverso multiplicativo de s módulo n :

$$w \equiv s^{-1} \pmod{n}$$

4. Calculam-se os coeficientes escalares:

$$u_1 \equiv e \cdot w \pmod{n}, \quad u_2 \equiv r \cdot w \pmod{n}$$

5. Computa-se a combinação linear de pontos na curva elíptica:

$$P = u_1 \cdot G + u_2 \cdot Q$$

6. A assinatura é **válida** se e somente se $P \neq \mathcal{O}$ e sua coordenada x coincidir com r :

$$x_P \equiv r \pmod{n}$$

Demonstração Algébrica Formal da Corretude

Por que o ponto P coincide exatamente com o ponto efêmero $R = k \cdot G$?

Substituindo $Q = d \cdot G$ na equação do ponto verificador P :

$$\begin{aligned} P &= u_1 \cdot G + u_2 \cdot Q \\ &= (e \cdot w) \cdot G + (r \cdot w) \cdot (d \cdot G) \\ &= (e \cdot w + r \cdot w \cdot d) \cdot G \\ &= w(e + r \cdot d) \cdot G \end{aligned}$$

Lembrando que $w \equiv s^{-1} \pmod{n}$:

$$w(e + r \cdot d) \equiv s^{-1}(e + r \cdot d) \pmod{n}$$

Da definição de s na assinatura:

$$s \equiv k^{-1}(e + r \cdot d) \pmod{n} \implies s^{-1} \equiv [k^{-1}(e + r \cdot d)]^{-1} \equiv k \cdot (e + r \cdot d)^{-1} \pmod{n}$$

Multiplicando por $(e + r \cdot d)$:

$$s^{-1}(e + r \cdot d) \equiv k \cdot (e + r \cdot d)^{-1} \cdot (e + r \cdot d) \equiv k \pmod{n}$$

Substituindo de volta na expressão do ponto P :

$$P = k \cdot G = R$$

Portanto, a coordenada x do ponto calculado P é exatamente x_R , que por construção é $r \pmod{n}$. A verificação aceita com 100% de consistência algébrica!

O Desastre Mortal: Reutilização do Nonce k

O nonce k é chamado de **número usado uma única vez (nonce)** por um motivo vital. Se o mesmo valor de k for reutilizado para assinar duas mensagens diferentes m_1 e m_2 com a mesma chave privada d :

Como $R = k \cdot G$, a coordenada r será **idêntica** em ambas as assinaturas: - Assinatura 1: (r, s_1) para a mensagem m_1 , com hash $e_1 = H(m_1)$ - Assinatura 2: (r, s_2) para a mensagem m_2 , com hash $e_2 = H(m_2)$

As duas equações de assinatura módulo n são:

$$s_1 \equiv k^{-1}(e_1 + r \cdot d) \pmod{n}$$

$$s_2 \equiv k^{-1}(e_2 + r \cdot d) \pmod{n}$$

Subtraindo a segunda equação da primeira:

$$s_1 - s_2 \equiv k^{-1}(e_1 - e_2) \pmod{n}$$

Como $e_1 \neq e_2$, o atacante calcula o inverso multiplicativo de $(s_1 - s_2)$ e **descobre o valor secreto de k** :

$$k \equiv (e_1 - e_2) \cdot (s_1 - s_2)^{-1} \pmod{n}$$

Uma vez que k é conhecido publicamente pelo invasor, ele isola a chave privada mestra d da primeira equação:

$$s_1 \cdot k \equiv e_1 + r \cdot d \pmod{n} \implies r \cdot d \equiv s_1 \cdot k - e_1 \pmod{n}$$

Multiplicando pelo inverso modular de r :

$$d \equiv (s_1 \cdot k - e_1) \cdot r^{-1} \pmod{n}$$

A chave privada d foi totalmente recuperada e o sistema foi destruído.

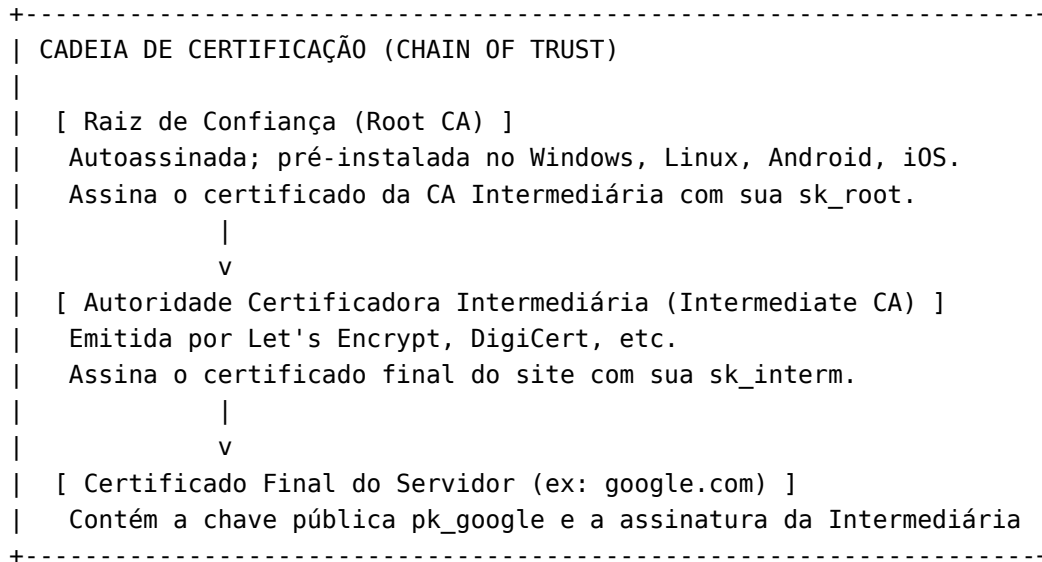
Incidentes Reais da História: 1. **Sony PlayStation 3 (2010):** A Sony implementou o ECDSA em seu bootloader gerando um k estático (fixo). O grupo hacker *fail0verflow* isolou a chave privada mestra da Sony em minutos, permitindo a execução de qualquer software no console. 2. **Bitcoin Android Wallet (2013):** Uma falha no gerador pseudoaleatório

SecureRandom do Java no Android gerou colisões de k em transações Bitcoin, permitindo o roubo automatizado de fundos.

Solução Moderna Definitiva: - **RFC 6979:** Gera o nonce k de forma **determinística**, derivando $k = \text{HMAC}(\text{sk}, H(m))$. Para a mesma mensagem o k é idêntico (o que não quebra nada), e para mensagens diferentes é impossível colidir. - **Ed25519 (EdDSA):** Algoritmo moderno sobre a Curve25519 que é determinístico por especificação nativa, imune a erros de gerador de números aleatórios e imune a ataques de canal lateral.

6.5 Certificados Digitais e Infraestrutura de Chaves Públicas (PKI)

Camada 1 — Visão Geral: Se Alice publicar uma chave pública pk_A , como Bob tem certeza de que aquela chave pertence a Alice e não a Mallory? A resposta é a **PKI (Public Key Infrastructure)** baseada em **Certificados Digitais X.509**.



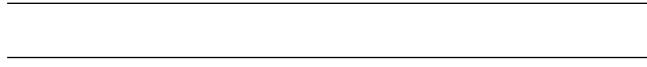
O Conteúdo de um Certificado X.509

Um certificado digital é uma estrutura padronizada contendo: - **Sujeito (Subject):** O nome do domínio/proprietário (ex: CN = api.banco.com). - **Chave Pública do Sujeito:** A chave pk pública real daquele domínio. - **Emissor (Issuer):** A Autoridade Certificadora que emitiu e atestou o documento. - **Validade:** Data inicial e data final de expiração (hoje tipicamente 90 dias). - **Assinatura Digital da CA:** $\sigma_{CA} = \text{Sign}(\text{sk}_{CA}, H(\text{dados do certificado}))$.

Revogação e Segurança Operacional

- **CRL (Certificate Revocation List):** Lista estática de certificados cancelados; pesada e ineficiente.
- **OCSP (Online Certificate Status Protocol):** Consulta em tempo real aos servidores da CA sobre o status do certificado.
- **OCSP Stapling:** O próprio servidor web consulta a CA periodicamente e envia uma prova com carimbo de tempo ao navegador, preservando a privacidade do usuário e eliminando lentidão.

- **Certificate Transparency (CT Logs):** Logs públicos imutáveis baseados em Árvores de Merkle onde todos os certificados TLS emitidos no mundo devem ser registrados obrigatoriamente, impedindo que CAs emitam certificados fraudulentos secretamente.



7. Curvas Elípticas (Vídeo V8)

7.1 O que é uma Curva Elíptica?

Camada 1 — Visão Geral: Uma curva elíptica não é uma elipse. O nome tem origem histórica no cálculo do comprimento de arco de elipses (integrais elípticas). Em criptografia, uma curva elíptica é o conjunto de soluções de uma equação cúbica suave de duas variáveis.

A Equação de Weierstrass

Na forma afim simplificada (válida para corpos com característica diferente de 2 e 3), a curva elíptica é definida pela equação:

$$y^2 = x^3 + ax + b$$

onde a e b são constantes que definem a curva.

A Condição do Discriminante (Sem Singularidades)

Para que a curva seja utilizável em criptografia, ela precisa ser **suave**, isto é, não pode conter pontos de singularidade (auto-intersecções com nós ou cúspides pontiagudas onde a derivada não existe). A condição necessária e suficiente é que o discriminante seja não-nulo:

$$\Delta = -16(4a^3 + 27b^2) \neq 0 \iff 4a^3 + 27b^2 \neq 0$$

Se $\Delta = 0$, a curva elíptica degenera e não é possível definir uma lei de grupo consistente sobre todos os seus pontos.



O Ponto no Infinito $\mathcal{O}$ e o Espaço Projetivo

No plano cartesiano afim $\mathbb{R}^2$, qualquer reta vertical paralela ao eixo y cruza a curva em no máximo dois pontos simétricos (x, y) e $(x, -y)$, nunca num terceiro ponto afim.

Para completar a geometria, introduz-se formalmente o **Ponto no Infinito**, denotado por $\mathcal{O}$. - No plano projetivo $\mathbb{P}^2$, as coordenadas homogêneas são $(X : Y : Z)$, e a equação se torna:

$$Y^2Z = X^3 + aXZ^2 + bZ^3$$

- Ao fazer $Z = 0$ (a reta no infinito), a equação fica $X^3 = 0 \implies X = 0$, restando apenas o ponto $(0 : 1 : 0)$, que é exatamente $\mathcal{O}$. - Todas as retas verticais são paralelas no plano afim, mas no plano projetivo elas convergem e se interceptam no ponto $\mathcal{O}$. - **Papel no Grupo:** $\mathcal{O}$ atua como o **Elemento Neutro Aditivo** do grupo da curva elíptica:

$$P + \mathcal{O} = P, \quad \forall P \in E$$

O Inverso de um Ponto: Simetria no Eixo x

Como a equação da curva possui apenas potências pares de y (y^2), se (x, y) pertence à curva, então $(x, -y)$ também pertence à curva. O inverso aditivo de um ponto $P = (x, y)$ é definido pela sua reflexão vertical:

$$-P = (x, -y)$$

A reta que une $P = (x, y)$ e $-P = (x, -y)$ é perfeitamente vertical e intercepta a curva no infinito $\mathcal{O}$. Logo:

$$P + (-P) = \mathcal{O}$$

7.2 A Lei de Grupo Geométrica (“Chord-and-Tangent”)

Os pontos de uma curva elíptica juntamente com $\mathcal{O}$ formam um **Grupo Abeliano** sob uma operação de adição geométrica denominada método da secante e tangente (*chord-and-tangent rule*).

Regra Fundamental da Adição

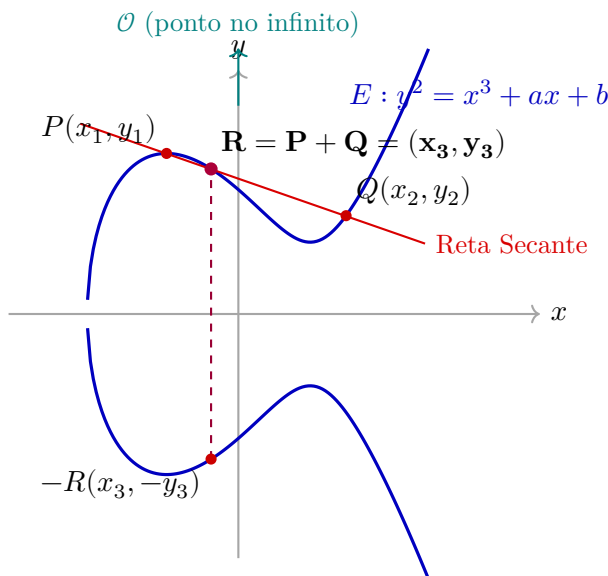
Axioma Geométrico: Se três pontos da curva elíptica são **colineares** (estão na mesma linha reta), a soma de suas coordenadas no grupo resulta no elemento neutro $\mathcal{O}$:

$$P + Q + (-R) = \mathcal{O} \implies P + Q = R$$

Para somar dois pontos distintos P e Q : 1. Traça-se a reta secante passando por P e Q . 2. Devido ao Teorema de Bézout, uma linha reta intercepta uma curva cúbica em exatamente **3 pontos** (contando multiplicidades). A reta cortará a curva num terceiro ponto, denotado por $-R = (x_3, -y_3)$. 3. Reflete-se esse terceiro ponto verticalmente sobre o eixo x para obter a soma:

$$R = P + Q = (x_3, y_3)$$

Ilustração Geométrica da Adição em Curvas Elípticas



Dedução Algébrica Completa das Fórmulas da Lei de Grupo

Vamos deduzir formalmente as equações para calcular $R = (x_3, y_3) = P + Q$:

1. A Inclinação λ da Reta:

- **Caso 1: Reta Secante** ($P \neq Q$ com $x_1 \neq x_2$):

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}$$

- **Caso 2: Reta Tangente / Duplicação** ($P = Q$ com $y_1 \neq 0$): Calcula-se a derivada da curva elíptica $y^2 = x^3 + ax + b$ por diferenciação implícita:

$$2y \frac{dy}{dx} = 3x^2 + a \implies \lambda = \frac{dy}{dx} = \frac{3x_1^2 + a}{2y_1}$$

2. Equação da Linha Reta: A reta que passa por $P(x_1, y_1)$ com inclinação λ é dada por:

$$y = \lambda(x - x_1) + y_1$$

3. Intersecção da Reta com a Curva Elíptica: Substitui-se a equação da reta na equação da curva $y^2 = x^3 + ax + b$:

$$(\lambda(x - x_1) + y_1)^2 = x^3 + ax + b$$

Expandindo e agrupando todos os termos em potências decrescentes de x :

$$x^3 - \lambda^2 x^2 + \dots = 0$$

4. Aplicação das Relações de Viète: As raízes desta equação polinomial cúbica são precisamente as abscissas dos três pontos de intersecção: x_1, x_2, x_3 . Pelas relações de Viète, a soma das três raízes de um polinômio cúbico mônico $x^3 - c_2 x^2 + c_1 x - c_0 = 0$ é igual ao coeficiente do termo quadrático:

$$x_1 + x_2 + x_3 = \lambda^2$$

Isolando a coordenada x_3 do terceiro ponto:

$$x_3 = \lambda^2 - x_1 - x_2$$

(Para duplicação $P = Q$, como $x_1 = x_2$, a fórmula torna-se $x_3 = \lambda^2 - 2x_1$).

5. A Coordenada y_3 (Reflexão Vertical): A ordenada da intersecção $-R$ na reta é:

$$y_{\text{inter}} = \lambda(x_3 - x_1) + y_1$$

Como $R = P + Q$ é a reflexão de $-R$ no eixo x , invertamos o sinal ($y_3 = -y_{\text{inter}}$):

$$y_3 = -[\lambda(x_3 - x_1) + y_1] = \lambda(x_1 - x_3) - y_1$$

7.3 Curvas Elípticas sobre Corpos Finitos ($\mathbb{F}_p$)

No mundo da computação e da criptografia, números reais $\mathbb{R}$ não podem ser usados devido a erros de arredondamento de ponto flutuante e vulnerabilidades de aproximação contínua. Em vez disso, a curva elíptica é construída sobre um **Corpo Finito** $\mathbb{F}_p$, onde p é um número primo grande:

$$y^2 \equiv x^3 + ax + b \pmod{p}$$

com $a, b \in \mathbb{F}_p$ e $4a^3 + 27b^2 \not\equiv 0 \pmod{p}$.

Propriedades da Curva em $\mathbb{F}_p$

- **Nuvem Discreta de Pontos:** O conjunto $E(\mathbb{F}_p)$ não forma mais uma curva contínua, mas sim uma matriz dispersa de pontos discretos $(x, y) \in \{0, \dots, p-1\}^2$, com simetria horizontal em torno de $y = p/2$.
- **Substituição de Divisão por Inverso Modular:** Toda divisão de inteiros nas fórmulas de λ é obrigatoriamente convertida em multiplicação pelo inverso multiplicativo modular $\pmod{p}$:

$$\lambda \equiv \begin{cases} (y_2 - y_1) \cdot (x_2 - x_1)^{-1} \pmod{p} & \text{se } P \neq Q \\ (3x_1^2 + a) \cdot (2y_1)^{-1} \pmod{p} & \text{se } P = Q \end{cases}$$

$$x_3 \equiv \lambda^2 - x_1 - x_2 \pmod{p}$$

$$y_3 \equiv \lambda(x_1 - x_3) - y_1 \pmod{p}$$

Ordem do Grupo e o Teorema de Hasse

O número total de pontos na curva, denotado pela ordem $\#E(\mathbb{F}_p)$ (incluindo o ponto $\mathcal{O}$), é finito. O matemático Helmut Hasse provou que a quantidade de pontos é sempre muito próxima de $p + 1$:

$$|\#E(\mathbb{F}_p) - (p + 1)| \leq 2\sqrt{p}$$

isto é:

$$p + 1 - 2\sqrt{p} \leq \#E(\mathbb{F}_p) \leq p + 1 + 2\sqrt{p}$$

7.4 Exemplo Numérico Completo Resolvido Passo a Passo

Vamos resolver as contas com a curva elíptica:

$$E : y^2 \equiv x^3 + 2x + 2 \pmod{17}$$

com parâmetros $a = 2$, $b = 2$ sobre o corpo $\mathbb{F}_{17}$.

Teste de Sanidade do Ponto Base $P = (5, 1)$

Verificando se $P = (5, 1)$ pertence a $E(\mathbb{F}_{17})$: - Lado esquerdo: $y^2 = 1^2 = 1 \pmod{17}$. - Lado direito: $x^3 + 2x + 2 = 5^3 + 2(5) + 2 = 125 + 10 + 2 = 137$. Dividindo 137 por 17: $137 = 8 \times 17 + 1 \equiv 1 \pmod{17}$. Como $1 \equiv 1 \pmod{17}$, o **ponto P pertence à curva** ✓.

Passo 1: Duplicação de Ponto ($2P = P + P$)

Como os pontos coincidem ($P = Q$), utilizamos a fórmula da reta tangente:

$$\lambda \equiv \frac{3x_1^2 + a}{2y_1} \equiv \frac{3(5^2) + 2}{2(1)} = \frac{3(25) + 2}{2} = \frac{77}{2} \pmod{17}$$

1. Redução do numerador: $77 = 4 \times 17 + 9 \implies 77 \equiv 9 \pmod{17}$. 2. Inverso multiplicativo modular de 2 (mod 17): Procuramos y tal que $2y \equiv 1 \pmod{17}$. Como $2 \times 9 = 18 = 17 + 1 \equiv 1 \pmod{17}$, temos $2^{-1} \equiv 9 \pmod{17}$. 3. Multiplicando o numerador pelo inverso modular do denominador:

$$\lambda \equiv 9 \times 9 = 81 \pmod{17}$$

Como $81 = 4 \times 17 + 13$, obtemos $\lambda \equiv 13 \pmod{17}$.

Agora calculamos as coordenadas (x_3, y_3) do ponto $2P$: - Coordenada x_3 :

$$x_3 \equiv \lambda^2 - 2x_1 = 13^2 - 2(5) = 169 - 10 = 159 \pmod{17}$$

Como $159 = 9 \times 17 + 6$, temos $x_3 \equiv 6 \pmod{17}$. - Coordenada y_3 :

$$y_3 \equiv \lambda(x_1 - x_3) - y_1 = 13(5 - 6) - 1 = 13(-1) - 1 = -14 \pmod{17}$$

Ajustando o resto negativo: $-14 + 17 = 3$, logo $y_3 \equiv 3 \pmod{17}$.

Resultado:

$$2P = (6, 3)$$

Conferindo na curva: $3^2 = 9 \pmod{17}$; e $6^3 + 2(6) + 2 = 216 + 12 + 2 = 230 = 13 \times 17 + 9 \equiv 9 \pmod{17}$ ✓.

Passo 2: Adição de Pontos Distintos ($3P = 2P + P$)

Somamos o ponto $2P = (6, 3)$ com o ponto $P = (5, 1)$ ($P \neq Q$, logo usamos a reta secante):

$$\lambda \equiv \frac{y_2 - y_1}{x_2 - x_1} \equiv \frac{1 - 3}{5 - 6} = \frac{-2}{-1} = 2 \pmod{17}$$

A inclinação da secante é simplesmente $\lambda \equiv 2 \pmod{17}$.

Calculando as novas coordenadas: - Coordenada x_3 :

$$x_3 \equiv \lambda^2 - x_1 - x_2 = 2^2 - 6 - 5 = 4 - 11 = -7 \pmod{17}$$

Ajustando o negativo: $-7 + 17 = 10$, logo $x_3 \equiv 10 \pmod{17}$. - Coordenada y_3 :

$$y_3 \equiv \lambda(x_1 - x_3) - y_1 = 2(6 - 10) - 3 = 2(-4) - 3 = -8 - 3 = -11 \pmod{17}$$

Ajustando o negativo: $-11 + 17 = 6$, logo $y_3 \equiv 6 \pmod{17}$.

Resultado:

$$3P = (10, 6)$$

Conferindo na curva: $6^2 = 36 = 2 \times 17 + 2 \equiv 2 \pmod{17}$; e $10^3 + 2(10) + 2 = 1000 + 20 + 2 = 1022 = 60 \times 17 + 2 \equiv 2 \pmod{17}$ ✓.

(Nota: Esta curva elíptica específica sobre $\mathbb{F}_{17}$ possui exatamente 19 pontos, sendo um grupo cíclico de ordem prima).

7.5 O Problema do Logaritmo Discreto em Curvas Elípticas (ECDLP)

Definição: Dado um ponto gerador G em uma curva elíptica e um ponto público resultante $Q = k \cdot G$, o **ECDLP (Elliptic Curve Discrete Logarithm Problem)** consiste em determinar o número inteiro escalar k .

- **Direção Fácil (Multiplicação Escalar):** Calcular $Q = k \cdot G$ é executado em tempo logarítmico $O(\log k)$ através do algoritmo **Double-and-Add** (duplica-se o ponto a cada bit e soma-se G se o bit for '1').
- **Direção Difícil (Reversão):** Encontrar k a partir de G e Q é computacionalmente intratável em curvas bem construídas.

Por que o ECC Usa Chaves Muito Menores que o RSA?

Esta é a pergunta central da criptografia moderna:

1. **Em $\mathbb{Z}_p^*$ (RSA e Diffie-Hellman clássico):** Os inteiros admitem fatoração única em números primos. Isso permitiu aos matemáticos inventarem o algoritmo de **Index Calculus** e o **General Number Field Sieve (GNFS)**, que encontram logaritmos discretos e fatoram números em tempo subexponencial:

$$L_p[1/3, c] = O\left(\exp\left(c \cdot (\ln p)^{1/3}(\ln \ln p)^{2/3}\right)\right)$$

Por essa razão, para manter 128 bits de segurança, o módulo RSA precisa crescer para gigantescos **3072 bits**.

2. **Em Curvas Elípticas $E(\mathbb{F}_p)$:** Pontos de uma curva elíptica **não possuem estrutura de fatoração** (não existe uma base de “pontos primos”). Portanto, **o algoritmo de Index Calculus NÃO se aplica a curvas elípticas**. Os melhores ataques conhecidos contra curvas elípticas gerais são **estritamente exponenciais**, limitados ao algoritmo genérico **Pollard ρ** , cujo custo computacional é:

$$O(\sqrt{n})$$

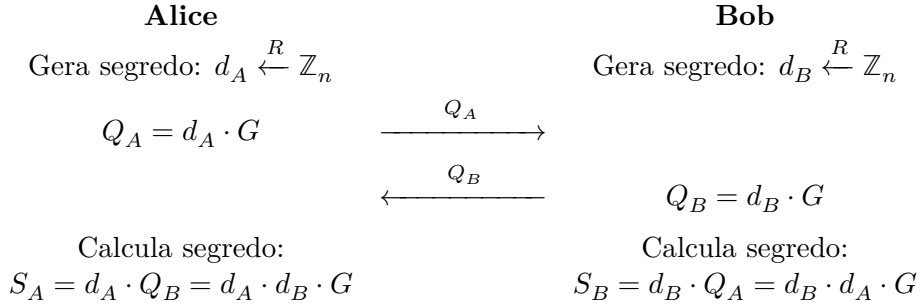
onde n é a ordem do subgrupo.

Conclusão Prática: Para atingir uma segurança de 2^{128} passos com Pollard ρ , precisamos apenas que $\sqrt{n} \approx 2^{128} \implies n \approx 2^{256}$. Uma curva de **256 bits** atinge os mesmos 128 bits de segurança que um módulo RSA de **3072 bits**!

7.6 Protocolos em Curvas: ECDH e Curvas Padrão

ECDH (Elliptic Curve Diffie-Hellman)

O protocolo de troca de chaves Diffie-Hellman em curvas elípticas é modelado diretamente com multiplicadores escalares:



Ambos chegam ao mesmo ponto secreto $S = (x_S, y_S)$. A chave simétrica da sessão é derivada da coordenada x_S :

$$k = \text{KDF}(x_S)$$

Curvas Mais Utilizadas no Mundo Real

1. **secp256k1**: Curva de Koblitz definida por $y^2 \equiv x^3 + 7 \pmod{p}$, com primos especiais que aceleram cálculos. Famosa por ser a curva do **Bitcoin, Ethereum e Blockchain**.
2. **NIST P-256 (secp256r1)**: Curva com coeficientes pseudoaleatórios padronizada pelo governo norte-americano, dominante em sites HTTPS, conexões TLS e certificados digitais governamentais.
3. **Curve25519 (X25519 / Ed25519)**: Criada pelo criptógrafo Daniel J. Bernstein (djb). É implementada na forma de Montgomery e Edwards. Foi projetada do zero para ser imune a falhas de canal lateral (timing attacks), cálculos sem inversões intermediárias e imune a curvas inválidas. Padrão no **WhatsApp, Signal, Tor, Apple iOS, SSH e WireGuard**.

8. Aprofundamentos (Contas na Mão)

8.1 Teorema Chinês do Resto (CRT) — Exemplo Numérico Completo

Problema: Encontrar o menor inteiro positivo x que satisfaça simultaneamente o sistema de congruências:

$$\begin{cases} x \equiv 2 \pmod{3} \\ x \equiv 3 \pmod{5} \\ x \equiv 2 \pmod{7} \end{cases}$$

Como os módulos $n_1 = 3, n_2 = 5, n_3 = 7$ são primos entre si aos pares ($\text{gcd} = 1$), o CRT garante solução única módulo $N = 3 \times 5 \times 7 = 105$.

Tabela de Resolução do CRT

i	Módulo n_i	Resto a_i	$N_i = N/n_i$	Inverso $y_i \equiv N_i^{-1}$ (mod n_i)	Parcela $a_i \cdot N_i \cdot y_i$
1	3	2	$105/3 = 35$	$35 \equiv 2$ (mod 3) $\Rightarrow$ $2 \times 2 = 4 \equiv$ $1 \Rightarrow \mathbf{y_1 = 2}$	$2 \times 35 \times 2 =$ 140
2	5	3	$105/5 = 21$	$21 \equiv 1$ (mod 5) $\Rightarrow$ $1 \times 1 = 1 \equiv$ $1 \Rightarrow \mathbf{y_2 = 1}$	$3 \times 21 \times 1 =$ 63
3	7	2	$105/7 = 15$	$15 \equiv 1$ (mod 7) $\Rightarrow$ $1 \times 1 = 1 \equiv$ $1 \Rightarrow \mathbf{y_3 = 1}$	$2 \times 15 \times 1 =$ 30

Recombinação e Redução

Somam-se todas as parcelas:

$$X = \sum_{i=1}^3 a_i \cdot N_i \cdot y_i = 140 + 63 + 30 = 233$$

Reduzindo módulo $N = 105$:

$$x \equiv 233 \pmod{105} \Rightarrow 233 = 2 \times 105 + 23 \Rightarrow \mathbf{x = 23}$$

Conferência das Congruências

- $23 = 7 \times 3 + 2 \Rightarrow 23 \equiv 2 \pmod{3} \checkmark$
- $23 = 4 \times 5 + 3 \Rightarrow 23 \equiv 3 \pmod{5} \checkmark$
- $23 = 3 \times 7 + 2 \Rightarrow 23 \equiv 2 \pmod{7} \checkmark$

8.2 RSA-CRT — Como a Decifragem Fica 4× Mais Rápida e o Ataque de Bellcore

Ao decifrar com RSA ou gerar assinaturas, deve-se computar $m \equiv c^d \pmod{N}$. Para N de 2048 bits, esta operação é computacionalmente pesada. O algoritmo RSA-CRT decompõe o cálculo em duas contas menores sobre os fatores primos p e q de 1024 bits.

Pré-computações Armazenadas na Chave Privada

- $d_p \equiv d \pmod{p-1}$
- $d_q \equiv d \pmod{q-1}$
- $q_{\text{inv}} \equiv q^{-1} \pmod{p}$

Algoritmo de Garner para RSA-CRT

1. Calcula-se a redução nos módulos primos:

$$m_1 \equiv c^{d_p} \pmod{p}, \quad m_2 \equiv c^{d_q} \pmod{q}$$

2. Recombina-se pelo algoritmo de Garner:

$$h \equiv q_{\text{inv}} \cdot (m_1 - m_2) \pmod{p}$$

$$m = m_2 + h \cdot q$$

Por que a Aceleração é de $4\times$?

A complexidade de tempo da exponenciação modular de k bits cresce cubicamente: $O(k^3)$. - Sem CRT: $(2048)^3 = 1$ unidade relativa de custo. - Com CRT: duas exponenciações de 1024 bits cada:

$$2 \times \left(\frac{1}{2}\right)^3 = 2 \times \frac{1}{8} = \frac{1}{4}$$

O tempo total cai para um quarto ($4\times$ mais rápido)!

O Ataque de Falha de Bellcore (Boneh, DeMillo, Lipton 1997)

O RSA-CRT introduz uma vulnerabilidade mortal se o hardware sofrer indução de falha física (*fault injection*), como um pico de tensão ou emissão laser durante o cálculo):

1. Suponha que o processador calcule $m_2 \pmod{q}$ com perfeição, mas sofra um erro em $m_1 \pmod{p}$, gerando um valor corrompido m'_1 .
2. A recombinação produz uma assinatura corrompida σ' tal que:

$$\sigma'^e \not\equiv m \pmod{p}, \quad \text{mas} \quad \sigma'^e \equiv m \pmod{q}$$

3. Isso significa que a diferença $(\sigma'^e - m)$ é um múltiplo do primo q , mas **não** é múltiplo de p !
4. O atacante calcula o Maior Divisor Comum com o módulo público N :

$$\gcd(\sigma'^e - m, N) = q$$

5. O cálculo do MDC via algoritmo de Euclides leva frações de milissegundo e **revela imediatamente o fator secreto** q , fatorando N e destruindo a chave privada inteira!

Contra-medida Obrigatória: Toda biblioteca séria (OpenSSL, BouncyCastle) valida a assinatura calculando $\sigma^e \stackrel{?}{\equiv} m \pmod{N}$ antes de devolvê-la ao usuário. Se houver falha, a assinatura é descartada.

8.3 Algoritmos de Quebra do Logaritmo Discreto

1. Baby-Step Giant-Step (BSGS) — Algoritmo de Shanks

Resolve $g^x \equiv h \pmod{p}$ em um grupo de ordem n . Baseia-se no compromisso espaço-tempo (*space-time tradeoff*).

1. Escreve-se o expoente desconhecido como:

$$x = i \cdot m + j, \quad \text{com } m = \lceil \sqrt{n} \rceil \text{ e } 0 \leq j, i < m$$

2. A equação $g^{i \cdot m + j} \equiv h \pmod{p}$ é rearranjada para:

$$g^j \equiv h \cdot (g^{-m})^i \pmod{p}$$

3. **Passos de Bebê (Baby Steps):** Calcula-se $g^j \bmod p$ para todos os $j \in \{0, 1, \dots, m-1\}$ e armazena-se na tabela hash os pares (g^j, j) .
4. **Passos de Gigante (Giant Steps):** Calcula-se $g^{-m} \equiv (g^m)^{-1} \pmod{p}$. Para cada $i = 0, 1, \dots, m-1$, computa-se $y \equiv h \cdot (g^{-m})^i \pmod{p}$ e verifica-se se y já existe na tabela hash. Ao encontrar colisão $g^j = y$, obtém-se imediatamente:

$$x = i \cdot m + j$$

- **Custo:** Tempo $O(\sqrt{n})$ e **Memória** $O(\sqrt{n})$.

2. Algoritmo Pollard ρ (Memória Constante $O(1)$)

O BSGS requer gigabytes de memória RAM para grupos médios. O algoritmo **Pollard ρ** atinge o mesmo tempo $O(\sqrt{n})$, mas consumindo **memória** $O(1)$, tornando-o o ataque mais perigoso contra ECDSA e ECDLP.

- **Mecânica:** Gera um passeio pseudoaleatório de pontos $P_k = a_k G + b_k Q$.
- Pelo **Paradoxo do Aniversário**, após aproximadamente $\approx 1,25\sqrt{n}$ passos, o passeio obrigatoriamente colide consigo mesmo, entrando em um ciclo com o formato geométrico da letra grega ρ (rho).
- A colisão é detectada em tempo real com o **Algoritmo dos Dois Ponteiros de Floyd** (um ponteiro avança 1 passo enquanto o outro avança 2 passos).
- Quando a colisão ocorre:

$$a_1 G + b_1 Q = a_2 G + b_2 Q \implies (a_1 - a_2)G = (b_2 - b_1)Q = (b_2 - b_1)dG$$

- Conclui-se o valor da chave secreta d :

$$d \equiv (a_1 - a_2) \cdot (b_2 - b_1)^{-1} \pmod{n}$$

9. Exercícios Resolvidos Passo a Passo

Resolva no papel antes de conferir a solução. Todas as respostas contêm o passo a passo algébrico completo e as congruências explicitadas.

Bloco A — Aritmética Modular e Grupos

1. Calcule $\gcd(1071, 462)$ utilizando o Algoritmo Euclidiano.
2. Encontre o inverso multiplicativo modular $17^{-1} \pmod{43}$ usando o Algoritmo Estendido de Euclides.
3. Quanto vale $\varphi(100)$? E quanto vale $\varphi(143)$?
4. Calcule $7^{222} \pmod{11}$ aplicando o Pequeno Teorema de Fermat.
5. Quantos elementos possui o grupo multiplicativo $\mathbb{Z}_{15}^*$? Liste todos os seus elementos. Ele é um grupo cíclico?
6. No grupo $\mathbb{Z}_{11}^*$, qual é a ordem do elemento $a = 3$? Ele é um gerador do grupo?

1. **Cálculo de $\gcd(1071, 462)$ por Euclides:**

- $1071 = 2 \times 462 + 147$
- $462 = 3 \times 147 + 21$
- $147 = 7 \times 21 + 0$ O último resto não-nulo é 21. Portanto, $\gcd(1071, 462) = 21$.

2. **Cálculo de $17^{-1} \pmod{43}$ via Euclides Estendido:**

- Divisões sucessivas:
 - $43 = 2 \times 17 + 9 \implies 9 = 43 - 2 \times 17$
 - $17 = 1 \times 9 + 8 \implies 8 = 17 - 1 \times 9$
 - $9 = 1 \times 8 + 1 \implies 1 = 9 - 1 \times 8$
- Substituição regressiva para expressar 1 como combinação linear:

$$1 = 9 - 1 \times (17 - 1 \times 9) = 2 \times 9 - 1 \times 17$$

$$1 = 2 \times (43 - 2 \times 17) - 1 \times 17 = 2 \times 43 - 5 \times 17$$

- Reduzindo módulo 43:

$$-5 \times 17 \equiv 1 \pmod{43}$$

Como $-5 \equiv -5 + 43 = 38 \pmod{43}$, temos:

$$17^{-1} \equiv \mathbf{38} \pmod{43}$$

Conferência: $17 \times 38 = 646 = 15 \times 43 + 1 \equiv 1 \pmod{43} \checkmark$.

3. **Cálculo da Função Totiente de Euler:**

- Para $100 = 2^2 \times 5^2$:

$$\varphi(100) = 100 \times \left(1 - \frac{1}{2}\right) \times \left(1 - \frac{1}{5}\right) = 100 \times \frac{1}{2} \times \frac{4}{5} = \mathbf{40}$$

- Para $143 = 11 \times 13$ (produto de dois primos distintos):

$$\varphi(143) = (11 - 1)(13 - 1) = 10 \times 12 = \mathbf{120}$$

4. **Cálculo de $7^{222} \pmod{11}$ por Fermat:**

- Como $p = 11$ é primo e $\gcd(7, 11) = 1$, pelo Pequeno Teorema de Fermat:

$$7^{10} \equiv 1 \pmod{11}$$

- Reduzimos o expoente no módulo da ordem $\pmod{10}$:

$$222 = 22 \times 10 + 2 \implies 222 \equiv 2 \pmod{10}$$

- Portanto:

$$7^{222} \equiv 7^2 = 49 \pmod{11}$$

Como $49 = 4 \times 11 + 5$, temos $7^{222} \equiv \mathbf{5} \pmod{11}$.

5. **Estrutura de $\mathbb{Z}_{15}^*$:**

- Como $15 = 3 \times 5$, temos $|\mathbb{Z}_{15}^*| = \varphi(15) = (3 - 1)(5 - 1) = 2 \times 4 = \mathbf{8}$ elementos.
- São os números em $\{1, \dots, 14\}$ coprimos com 15:

$$\mathbb{Z}_{15}^* = \{1, 2, 4, 7, 8, 11, 13, 14\}$$

- Pelo Teorema Chinês do Resto, $\mathbb{Z}_{15}^* \cong \mathbb{Z}_3^* \times \mathbb{Z}_5^* \cong \mathbb{Z}_2 \times \mathbb{Z}_4$.

- A ordem máxima de qualquer elemento é $\text{mmc}(2, 4) = 4 < 8$. Nenhum elemento tem ordem 8. Logo, $\mathbb{Z}_{15}^*$ **NÃO é cíclico**.
6. **Ordem de 3 em $\mathbb{Z}_{11}^*$** : Calculamos as potências sucessivas de 3 (mod 11):
- $3^1 \equiv 3 \pmod{11}$
 - $3^2 \equiv 9 \pmod{11}$
 - $3^3 \equiv 27 \equiv 5 \pmod{11}$
 - $3^4 \equiv 5 \times 3 = 15 \equiv 4 \pmod{11}$
 - $3^5 \equiv 4 \times 3 = 12 \equiv 1 \pmod{11}$ O menor expoente positivo que resulta em 1 é 5. Portanto, a ordem de 3 é $\text{ord}(3) = 5$. Como o grupo tem ordem $|\mathbb{Z}_{11}^*| = 10$ e $5 \neq 10$, 3 **não é gerador** de $\mathbb{Z}_{11}^*$ (ele gera apenas o subgrupo $\{1, 3, 4, 5, 9\}$).
-

Bloco B — RSA

- Sejam $p = 5$, $q = 11$ e expoente público $e = 3$. Determine o módulo N , o totiente $\varphi(N)$ e a chave privada d .
- Utilizando a chave obtida no exercício 7, cifre a mensagem $m = 9$ e em seguida decifre o texto cifrado resultante para recuperar m .
- Alice publicou $N = 3233$ e $e = 17$. Um espião descobriu que N fatora em $p = 61$ e $q = 53$. Calcule o expoente de decifragem privado d .
- Por que as implementações comerciais de RSA adotam universalmente $e = 65537$ em vez de $e = 3$?
- Demonstre formalmente a maleabilidade do RSA cru: dado um texto cifrado $c \equiv m^e \pmod{N}$, mostre como gerar uma cifra válida para $2m$ sem conhecer o texto claro m .

Respostas e Resoluções Detalhadas — Bloco B

7. Setup das Chaves RSA:

- Módulo: $N = p \cdot q = 5 \times 11 = \mathbf{55}$.
- Totiente: $\varphi(N) = (5 - 1)(11 - 1) = 4 \times 10 = \mathbf{40}$.
- Chave privada d : buscamos d tal que $3d \equiv 1 \pmod{40}$. Pelo algoritmo estendido: $3 \times 27 = 81 = 2 \times 40 + 1 \equiv 1 \pmod{40}$. Portanto, $d = 27$.

8. Cifragem e Decifragem:

- **Cifragem de $m = 9$:**

$$c \equiv m^e \equiv 9^3 = 729 \pmod{55}$$

Dividindo 729 por 55: $729 = 13 \times 55 + 14 \implies \mathbf{c = 14}$.

- **Decifragem de $c = 14$:**

$$m \equiv c^d \equiv 14^{27} \pmod{55}$$

Decompondo o expoente $27 = 16 + 8 + 2 + 1$:

- $14^1 \equiv 14 \pmod{55}$
- $14^2 = 196 = 3 \times 55 + 31 \equiv 31 \equiv -24 \pmod{55}$
- $14^4 \equiv (-24)^2 = 576 = 10 \times 55 + 26 \equiv 26 \pmod{55}$
- $14^8 \equiv 26^2 = 676 = 12 \times 55 + 16 \equiv 16 \pmod{55}$
- $14^{16} \equiv 16^2 = 256 = 4 \times 55 + 36 \equiv 36 \equiv -19 \pmod{55}$ Multiplicando os termos:

$$14^{27} = 14^{16} \times 14^8 \times 14^2 \times 14^1 \equiv (-19) \times 16 \times 31 \times 14 \pmod{55}$$

$$(-19) \times 16 = -304 = -6 \times 55 + 26 \equiv 26 \pmod{55}. \quad 31 \times 14 = 434 = 7 \times 55 + 49 \equiv -6 \pmod{55}. \quad m \equiv 26 \times (-6) = -156 = -3 \times 55 + 9 \equiv \mathbf{9} \pmod{55} \checkmark.$$

9. Chave Privada d para $N = 3233$, $e = 17$:

- $\varphi(N) = (61 - 1)(53 - 1) = 60 \times 52 = 3120$.
- Calculamos $d \equiv 17^{-1} \pmod{3120}$ via Euclides estendido:
 - $3120 = 183 \times 17 + 9 \implies 9 = 3120 - 183 \times 17$
 - $17 = 1 \times 9 + 8 \implies 8 = 17 - 1 \times 9$
 - $9 = 1 \times 8 + 1 \implies 1 = 9 - 1 \times 8 = 2 \times 9 - 1 \times 17 = 2(3120 - 183 \times 17) - 17$

$$1 = 2 \times 3120 - 367 \times 17$$

- Reduzindo mod 3120:

$$d \equiv -367 \equiv -367 + 3120 = \mathbf{2753} \pmod{3120}$$

10. **Por que $e = 65537$ e não $e = 3$:**

- O expoente $e = 3$ é vulnerável ao ataque de raiz direta se $m^3 < N$ e ao ataque de difusão de Håstad (basta interceptar a mesma mensagem enviada para 3 destinatários com $e = 3$).
- $e = 65537 = 2^{16} + 1$ é um número primo de Fermat com apenas dois bits ‘1’ em binário (10000000000000001_2). Ele requer apenas 16 operações de quadrado e 1 multiplicação modular no algoritmo square-and-multiply, aliando eficiência máxima à imunidade contra ataques de expoente pequeno.

11. **Demonstração Algébrica da Maleabilidade do RSA Cru:**

- Seja $c \equiv m^e \pmod{N}$.
- O adversário calcula a cifra de 2: $c_2 \equiv 2^e \pmod{N}$.
- Ele multiplica as duas cifras:

$$c' \equiv c \cdot c_2 \equiv m^e \cdot 2^e \equiv (2m)^e \pmod{N}$$

- Quando o receptor decifra c' com a chave privada d :

$$\text{Dec}(c') \equiv (c')^d \equiv ((2m)^e)^d \equiv 2m \pmod{N}$$

- O adversário obteve um cifrado válido para a mensagem dobrada $2m$ sem nunca saber quem era m .

Bloco C — Diffie-Hellman, ElGamal e Assinaturas

12. No protocolo Diffie-Hellman com parâmetros públicos $p = 23$ e $g = 5$, Alice escolhe segredo $a = 4$ e Bob escolhe $b = 3$. Qual é o segredo compartilhado final s ?
13. O que um espião passivo (Eve) observa no canal durante a troca do exercício 12? Por que Eve é incapaz de descobrir s ?
14. No criptosistema ElGamal, suponha que Alice cifre duas mensagens distintas m_1 e m_2 para Bob reutilizando o mesmo nonce efêmero r . Mostre como Eve, conhecendo apenas m_1 , recupera a mensagem secreta m_2 .
15. Um servidor cometeu a falha de assinar duas mensagens distintas m_1 e m_2 com ECDSA usando o mesmo nonce efêmero k , gerando assinaturas (r, s_1) e (r, s_2) . Demonstre como isolar k e em seguida extrair a chave privada mestra d .
16. Por que o paradigma Hash-and-Sign exige estritamente resistência à colisão para a função H , e não apenas resistência à pré-imagem?

Respostas e Resoluções Detalhadas — Bloco C

12. **Cálculo do Segredo DH:**

- Alice envia: $A \equiv 5^4 = 625 = 27 \times 23 + 4 \equiv \mathbf{4} \pmod{23}$.
- Bob envia: $B \equiv 5^3 = 125 = 5 \times 23 + 10 \equiv \mathbf{10} \pmod{23}$.
- Alice calcula: $s \equiv B^A \equiv 10^4 = 10000 = 434 \times 23 + 18 \equiv \mathbf{18} \pmod{23}$.
- Bob calcula: $s \equiv A^B \equiv 4^3 = 64 = 2 \times 23 + 18 \equiv \mathbf{18} \pmod{23} \checkmark$.

13. **Visão de Eve e Hipótese CDH:**

- Eve intercepta: $p = 23, g = 5, A = 4, B = 10$.
- Para achar o segredo $s = g^{ab} \pmod{p}$, Eve precisaria resolver o **Problema Computacional de Diffie-Hellman (CDH)** ou calcular o logaritmo discreto $a = \log_g A \pmod{p}$. Com parâmetros reais de 3072 bits em $\mathbb{Z}_p^*$ ou 256 bits em curvas elípticas, este cálculo exigiria centenas de anos dos maiores supercomputadores do planeta.

14. **Ataque de Reuso de Nonce no ElGamal:**

- As cifras geradas são:

$$c_1 \equiv g^r \pmod{p}, \quad c_2^{(1)} \equiv m_1 \cdot h^r \pmod{p}, \quad c_2^{(2)} \equiv m_2 \cdot h^r \pmod{p}$$

- Como o termo mascarador $h^r \pmod{p}$ é idêntico em ambas, Eve divide $c_2^{(2)}$ por $c_2^{(1)}$:

$$\frac{c_2^{(2)}}{c_2^{(1)}} \equiv \frac{m_2 \cdot h^r}{m_1 \cdot h^r} \equiv \frac{m_2}{m_1} \pmod{p}$$

- Conhecendo m_1 , Eve multiplica pelo inverso de m_1 :

$$m_2 \equiv m_1 \cdot c_2^{(2)} \cdot (c_2^{(1)})^{-1} \pmod{p}$$

- A mensagem secreta m_2 é desmascarada sem que Eve conheça a chave privada x .

15. **Quebra de Chave ECDSA por Repetição de k :**

- As assinaturas satisfazem:

$$s_1 \equiv k^{-1}(e_1 + r \cdot d) \pmod{n}, \quad s_2 \equiv k^{-1}(e_2 + r \cdot d) \pmod{n}$$

- Subtraindo as duas congruências:

$$s_1 - s_2 \equiv k^{-1}(e_1 - e_2) \pmod{n} \implies k \equiv (e_1 - e_2) \cdot (s_1 - s_2)^{-1} \pmod{n}$$

- Uma vez descoberto k , isola-se d da primeira equação:

$$s_1 \cdot k \equiv e_1 + r \cdot d \pmod{n} \implies d \equiv (s_1 \cdot k - e_1) \cdot r^{-1} \pmod{n}$$

16. **Necessidade de Resistência à Colisão:**

- Se a função não for resistente à colisão, um fraudador pode encontrar computacionalmente dois documentos diferentes com o mesmo hash: $H(m_{\text{inocente}}) = H(m_{\text{fraude}})$.
- Ele induz a vítima a assinar m_{inocente} , obtendo $\sigma = \text{Sign}(\text{sk}, H(m_{\text{inocente}}))$.
- Como os hashes colidem, σ torna-se uma assinatura matematicamente legítima para o documento fraudulento m_{fraude} , permitindo a transferência ilegal de valores ou forja de autoridade.

Bloco D — Perguntas Conceituais de Prova

17. Por que a relação $\varphi(N) = (p-1)(q-1)$ é denominada a “porta dos fundos” (*trapdoor*) do RSA?
18. Qual é a diferença prática fundamental entre segurança IND-CPA e IND-CCA em um cenário corporativo real?

19. Por que a Hipótese Decisional de Diffie-Hellman (DDH) é **falsa** no grupo multiplicativo $\mathbb{Z}_p^*$ inteiro, e como os padrões criptográficos contornam essa limitação?
20. Por que uma chave de Curva Elíptica de 256 bits (ex: secp256k1) atinge o mesmo nível de segurança computacional que um módulo RSA de 3072 bits?

Respostas Conceituais — Bloco D

17. **A Trapdoor do RSA:** Multiplicar dois primos p e q para obter $N = pq$ é uma operação fácil; recuperar p e q a partir de N exige fatoração (problema computacionalmente difícil). O conhecimento prévio dos fatores secretos p e q permite computar $\varphi(N) = (p-1)(q-1)$ instantaneamente, e através de $\varphi(N)$ inverter o expoente público para calcular $d \equiv e^{-1} \pmod{\varphi(N)}$. Quem conhece apenas N é incapaz de calcular $\varphi(N)$ sem antes fatorar o módulo.
18. **CPA vs. CCA no Mundo Real:**
 - **IND-CPA:** Protege contra invasores puramente **passivos**, que interceptam mensagens cifradas na rede e tentam correlacioná-las com textos conhecidos.
 - **IND-CCA:** Protege contra invasores **ativos**, que manipulam pacotes cifrados e os submetem a servidores, analisando tempos de resposta e mensagens de erro (oráculos de decifragem). No ataque de Bleichenbacher, o servidor agia como um oráculo de padding: o esquema era passivamente seguro, mas caiu ativamente sob CCA. Em sistemas de produção, IND-CCA2 é o requisito mínimo indispensável.
19. **Falha da DDH em $\mathbb{Z}_p^*$ e o Uso de Subgrupos:** No grupo $\mathbb{Z}_p^*$ inteiro, o Símbolo de Legendre / resíduo quadrático vaza 1 bit de informação determinístico sobre se um elemento é um quadrado perfeito mod p . Como o produto de dois não-resíduos é um resíduo, o adversário consegue distinguir g^{ab} de um elemento aleatório com vantagem não-desprezível de $1/2$. A solução padronizada consiste em trabalhar exclusivamente dentro de um **subgrupo de ordem prima** q de $\mathbb{Z}_p^*$ (gerado a partir de primos seguros $p = 2q + 1$), onde todos os elementos pertencem ao mesmo subgrupo de resíduos quadráticos e a DDH se sustenta rigorosamente.
20. **ECC 256 bits vs. RSA 3072 bits:** Em $\mathbb{Z}_p^*$, os elementos inteiros possuem fatoração em fatores primos, o que permite o funcionamento do algoritmo **Index Calculus / GNFS**, que quebra o logaritmo discreto e fatora módulos em tempo **subexponencial**. Em curvas elípticas, os pontos da curva não possuem estrutura de fatoração primária; os únicos algoritmos conhecidos contra o ECDLP são genéricos e **estritamente exponenciais** (como o algoritmo Pollard ρ , de complexidade $O(\sqrt{n})$). Assim, para obter 128 bits de segurança (2^{128} operações), o RSA precisa de um módulo de 3072 bits para compensar o avanço subexponencial, enquanto as curvas elípticas precisam de uma ordem prima de apenas $n \approx 2^{256}$ bits ($\sqrt{2^{256}} = 2^{128}$).

10. Colinhas Finais de Consulta Rápida

Mapa de Esquemas, Grupos e Hipóteses

Esquema	Grupo / Estrutura	Hipótese de Dificuldade	Função Principal	Padrão Atual Recomendado
RSA	Anel modular $\mathbb{Z}_N^*$ com $N = pq$	Fatoração Inteira / RSA Problem	Cifragem e Assinatura	RSA-OAEP / RSA-PSS (≥ 3072 bits)

Esquema	Grupo / Estrutura	Hipótese de Dificuldade	Função Principal	Padrão Atual Recomendado
DHKE / ECDH	Subgrupo de $\mathbb{Z}_p^*$ ou Curva $E(\mathbb{F}_p)$	CDH / DDH / ECDLP	Troca de Chaves Segura	X25519 (Curve25519) ou ECDH P-256
ElGamal	Subgrupo de ordem prima q de $\mathbb{Z}_p^*$	DDH (Decisional Diffie-Hellman)	Cifragem de Chave Pública	Substituído por HPKE / ECIES
DSA / ECDSA	Curva elíptica $E(\mathbb{F}_p)$ de ordem prima n	DL / ECDLP	Assinatura Digital	Ed25519 ou ECDSA com RFC 6979

Os 7 Erros Clássicos de Implementação

1. **RSA sem Padding (Textbook RSA):** Cifragem determinística (não-CPA) e multiplicativamente maleável, além de vulnerável a ataques de raiz e broadcast.
2. **Reutilização de Nonce (k) no ECDSA:** Permite isolar k e extrair a chave privada mestra d por subtração direta de duas assinaturas.
3. **Diffie-Hellman sem Autenticação:** Suscetível a ataques ativos de Man-in-the-Middle (MITM); deve ser obrigatoriamente associado a certificados digitais no TLS 1.3.
4. **Confundir Cifragem com Assinatura:** Cifrar protege o segredo da mensagem (usa a chave pública do destinatário); assinar garante autoria e integridade (usa a chave privada do remetente).
5. **Geradores Fracos de Números Aleatórios (RNG):** Usar funções randômicas padrão de biblioteca (`rand()`, `Math.random()`) em vez de geradores criptográficos (`/dev/urandom`, `getrandom()`).
6. **Oráculos de Padding por Resposta de Erro:** Tratar erros de padding revelando detalhes ao cliente, viabilizando o ataque de Bleichenbacher (ROBOT).
7. **Comparações de Chaves em Tempo Não-Constante:** Utilizar operadores convencionais de igualdade (`==`, `memcmp`) que encerram a checagem no primeiro byte divergente, abrindo brechas para ataques de canal lateral baseados em tempo (*timing attacks*).

Referências Bibliográficas Fundamentais

- **Katz, Jonathan; Lindell, Yehuda.** *Introduction to Modern Cryptography*. 3ª Edição, CRC Press, 2020. (Referência para definições formais de jogos CPA, CCA e EUF-CMA).
- **Menezes, Alfred J.; van Oorschot, Paul C.; Vanstone, Scott A.** *Handbook of Applied Cryptography*. CRC Press, 1996. (Disponível gratuitamente online; referência para algoritmos de teoria dos números).
- **Hoffstein, Jeffrey; Pipher, Jill; Silverman, Joseph H.** *An Introduction to Mathematical Cryptography*. Springer, 2ª Edição, 2014. (Referência definitiva para a geometria de curvas elípticas e corpos finitos).
- **Smart, Nigel.** *Cryptography Made Simple*. Springer, 2016. (Excelente equilíbrio entre álgebra abstrata e protocolos do mundo real).